République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene

Faculté de Mathématiques

Département de Recherche Opérationnelle

Mémoire

En vue de l'obtention du Diplôme de MASTER
en Recherche Opérationnelle Intitulé du master Management, Risque et Négociation

Thème

Modèle d'aide à la décision pour le système d'implantation des dépôts carburants

Présenté par : DEHILIS Nour El Houda
FERHI Feriel

Soutenu le : 27/06/2024, devant le jury composé de :

Président :	M^{r} $AIDER, M.$	$USTHB$
Examinatrice :	Mme $HADJ\ BENALI, L.K.$	$USTHB$
Rapporteur :	M^{r} $BENKANOUN, Y.$	$USTHB$
Invité :	M^{r} $.GHERRAMOU, S$	$NAFTAL$

Code de mémoire : 05/ROMARIN/2024

Au nom d'Allah, le Tout Miséricordieux, le Très Miséricordieux

بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ

In the name of Allah, the Most Gracious, the Most Merciful

Remerciements

Ce mémoire ne serait pas aussi riche et n'aurait pas pu avoir le jour sans l'aide et l'encadrement de Monsieur **BENKANOUN ,Y**.
On le remercie pour la qualité de son encadrement exceptionnel , pour sa patience , son éffort et sa disponibilité dans les moments les plus délicates .

Nous sommes conscients de l'honneur que nous a fait Monsieur **AÏDER ,M** .En étant notre proffeseur pendant 2 ans avant d'être le président du jury et Madame **HADJ BENALI ,L.K** d'avoir accepté d'examiner ce travail .

Nos reconnaissances s'adressent aussi à toute l'équipe de NAFTAL et spécialement notre promoteur Monsieur **GHERRAMOU,S** pour ses judicieux conseils et son aide lors de la réalisation de notre étude sur le terrain .

Un grand merci à Madame **Dahmani ,I**. pour tout ce qu'elle a fait , elle nous a guidé et nous a beaucoup aidé .

Nos remerciements s'étendent également à toutes les personnes qui ont participé de près ou de loin .

Dédicaces

À mon précieux offre du dieux ,ma force ,ma joie mon bras droit mon père ,les mots ne suffisent pas pour te remercier je suis chanceuse d'avoir un tel pére en or, qui a souffert sans me laisser souffrir,je n'oublierai jamais ce que vous avez fait pour moi .

À ma source d'inspiration, mon étoile,ma merveilleuse mère qui a prié pour moi chaque jour,a fait son possible pour me rendre heureuse ta présence est mon confort quotidien merci pour toutes tes sacrifices une telle maman c'est pas tout le monde qui la.

À ma sœur Manel.D ma meilleure amie ma boule d'énergie ma complice et ma confidente qui me remonte le moral ,ta gentillesse, ta bienveillance sont rares je suis fière de t'avoir.

À mon amie Feriel .K , pour toutes les fois où tu as été là pour moi, pour chaque sourire et chaque moment partagé merci du fond du coeur ,ta présence est le plus beau des cadeaux .

Je suis ici aujourd'hui grâce à votre encouragement et votre motivation.

Aucune dédicace ne pourra exprimer mes respects mes considérations et ma grande admiration pour vous.

Nour El Houda.

À ma très chère mère, quoi que je fasse ou que je dise, je ne saurais jamais te remercier comme il se doit.

À mon cher père, celui qui m'apporte soutien et guidance dans la vie, ton affection me couvre, ta bienveillance me guide, et ta présence à mes côtés a toujours été ma source de force pour affronter les différents obstacles.

À mes sœurs, Sarra, Samra, et Loubna, compagnes de mes rires et gardiennes de mes secrets.

À Mohammed et Abd Asalam, mes frères et mes piliers.

À mes amies depuis le lycée jusqu'a aujourd'hui.

À mes chers neveux Assil, Oussama et Djawad icarnations del'innocence et de la joie dans notre famille.

À tous mes amis et toutes les personnes qui ont croisé mon chemin.

Feriel.

Résumé

Ce travail fait partie d'un projet de fin d'études,dont l'objectif est de développer un nouveau schéma de distribution en éliminant les centres secondaires ,et en se référant aux centres primaires pour accroître leurs capacités de stockage en trouvant le meilleur emplacement des bacs ,Le plus court chemin entre l'entrepôt et la station service et donc une durée minimale pour la livraison .
Pour atteindre ces objectifs,nous avons résolu un problème bi-objectif à l'aide des heuristiques et des méta-heuristiques , plus précisément nous avons appliqué l'algorithme génétique de tri non dominé II que nous avons ensuite implémenté en python , les résultats obtenus sont satisfaisants.

Mots clés : Capacités de stockage,emplacement des bacs, le plus court chemin ,l'algorithme génétique de tri non dominé II,Python .

Abstract

This work is a part of the end of study project, whose objective is to develop a new distribution scheme by eliminating secondary centers ,and referring to primary centers to increase their storage capacities by finding the best bin location ,The shortest path between the warehouse and the station and therefore a minimum time for delivery .
To achieve these objectives, we solved a bi-objective problem using heuristics and meta-heuristics. Specifically, we applied the Non Dominated Sorting Genetic Algorithm II, which we then implemented in Python,the results achieved are satisfactory.

Keywords : Storage capacities, bin location,The shortest path,the Non Dominated Sorting Genetic Algorithm II,Python.

Liste des abréviations

Tout le long du mémoire, les règles d'abréviation suivantes ont été adoptées :

SONATRACH :Société Nationale de Transport et de la commercialisation des Hydrocarbures

NAFTAL : Société Nationale de la Commercialisation et la Distribution des produits pétroliers.

GPL :gaz de pétrole liquéfié

ESP :Essence Sans Plomb

GO :gasoil

CDS :centre de Stockage

CDD :centre de distribution

SBA : Sidi Bel Abbès

HMD : Hassi Messaoud

TAM : Tamnrasset

Tizi : Tizi Ouzou

BBA : Bordj Bou Arreridj

BBM :Bordj Badji Mokhtar

OEB :Oum el Bouaghi

PL :Programme linéaire

PMO :Problème multi-objectif

PBO :Problème bi-objectif

PERS :Problème d'emplacement des reservoirs de stockage

Table des matières

Bibliographie **1**

Introduction **15**

1 **18**

1.1 Introduction 18
1.2 Organisation générale de NAFTAL : 18
1.2.1 Direction générale : 19
1.2.2 Structures fonctionnelles : 19
1.3 Les activités du NAFTAL : 20
1.4 Les produits commercialisés par NAFTAL 21
1.4.1 Les carburants 21
1.4.2 Les lubrifiants (huiles et graisses) 22
1.4.3 Les Pneumatiques 22
1.4.4 Le bitume 22
1.4.5 Le fuel (léger et lourd) 22
1.5 Branche carburant 22
1.5.1 Localisation géographique 22
1.5.2 Présentation de la branche carburant 22
1.6 Les modes de transport 23
1.6.1 Les moyens fixes 23
1.6.2 Les moyens mobiles 23
1.7 La gestion des stocks 24
1.7.1 Les centres de stockage 24
1.8 Le réseau de distribution 24
1.8.1 L'approvisionement 25
1.8.2 Le ravitaillement 25
1.8.3 La livraison 25
1.9 Schéma de distribution actuel 26
1.9.1 La production des raffineries 26

1.10 Problématique : 31
1.11 Les objectifs : 31
1.12 Conclusion 31

2 **33**
2.1 Définition du problème 33
2.2 Programme linéaire 33
2.3 Problème d'optimisation 35
2.4 Solution réalisable et solution optimale 35
2.5 Problème d'optimisation combinatoire 36
2.6 Les classes de complexité 36
2.6.1 La classe P : 36
2.6.2 La classe NP(Non-déterministe Polynomial) : 36
2.6.3 La classe NP-Complet : 36
2.6.4 La classe NP-difficile : 37
2.7 Quelques problèmes d'optimisation combinatoire 37
2.8 Optimisation mono ou multi-objectif 38
2.9 Formulation générale d'un Problème multi-objectif 38
2.10 La multiplicité des solutions 39
2.11 Propriétés 40
2.11.1 Efficacité et dominance 40
2.11.2 Solution efficace 40
2.11.3 La relation de dominance 40
2.11.4 Les points caractéristiques 40

3 **43**
3.1 Introduction 43
3.2 Exemple Introductif 43
3.3 Position du problème 44
3.4 Caractéristiques du PERS 47
3.5 La formulation mathématique 47
3.6 Modèle mathématique 50
3.7 Évaluation du modèle 50
3.8 Validation du modèle 51
3.9 Conclusion 55

4 **57**
4.1 Introduction 57

4.2 Approche de résoultion des problèmes(PBO) 57
4.3 Les méthodes exactes 57
4.3.1 Procédure de séparation et d'évaluation 58
4.3.2 Les approches non Pareto 58
4.3.3 Méthode approchée 60
4.4 Les approches Pareto 61

5 **67**
5.1 Introduction 67
5.2 Environnement Informatique 67
5.3 L'outil Google Earth PRO 67
5.4 Les coordonnées géographiques des entrepôts et des stations services 68
5.5 Algorithme d'Attribution Stations-Entrepôts 70
5.6 Approche gloutonne 70
5.7 Les étapes clés de cet algorithme 73
5.8 Les étapes clés de cet algorithme 74
5.9 Perturabation de la solution 76
5.10 Python 77
5.11 Les bibliothèques utilisées 78
5.12 Autres Méthodes proposées 81

Annexes **85**

Liste des tableaux

1.1 Les réservoirs de stockage des centres secondaires 25
1.2 Liste des raffineries 26

3.1 Exemple d'un entrepôt avec une faible autonomie 43

5.1 Les rampes du sud avec leurs coordonnées géographiques 68
5.2 Les rampes et les entrepôts du nord avec leurs coordonnées géographiques 69
5.3 La demande dans la région SUD 70

Table des figures

1.1 Logo de l'entreprise NAFTAL. 18
1.2 Organisation générale du NAFTAL 20
1.3 Les activités du NAFTAL. 21
1.4 Flux de distribution 26
1.5 Clé des Schémas 28
1.6 Schéma de distribution de la région EST en démarrant par l'approvisionnement par pipe -line 28
1.7 Schéma de distribution de la région EST en démarrant par l'approvisionnement par cabotage 29
1.8 Schéma de distribution de la région CENTRE 29
1.9 Schéma de distribution de la région OUEST 30
1.10 Schéma de distribution de la région SUD 30

2.1 La relation entre les classes du problème 37
2.2 Représentation de l'espace des décisions et l'espaces des objectifs correspondant. 39

3.1 Exemple sur la livraison en droiture (Entrepôt SBA -> Réseau) . . . 44
3.2 Autonomie du GO sur le térritoire national 45
3.3 Autonomie d'ESP sur le térritoire national 45
3.4 Logo Cplex Optimizer 52

4.1 Cycle génétique 62
4.2 Les caractéristiques du front de Pareto 63
4.3 schéma récapitulatif des méthodes de résolution des problèmes multi-objectif 65

5.1 Exemple de longitude et latitude (X et Y respectivement) de la rampe HMD 68
5.2 Exemple de longitude et latitude d'une station service à Chlef 68
5.3 solution initiale d'approvisionnment 76
5.4 Python Logo 77
5.5 Le classement des langages de programmation en 2024 78

5.6 code d'installation des bacs 86
5.7 Code d'approvisionnement 87
5.8 Code d'attribution stations-entrepôts 91

Liste des algorithmes

1 Algorithme pour attribuer les stations-services aux entrepôts pour une région donnée 70
2 GreedyAlgorithm 72
3 Gestion du stock de la rampe 73
4 Approvisionnement de la raffinerie vers l'entrepôt 75

Introduction générale

La Recherche Opérationnelle (RO) est une discipline scientifique qui vise à aider le décideur à prendre de bonnes décisions en utilisant des modèles mathématiques, des techniques analytiques et des méthodes algorithmiques pour trouver les meilleures solutions possibles aux problèmes opérationnels. Dans le domaine industriel, les entreprises sont confrontées à de nombreux problèmes, parmi lesquels le problème d'emplacement des réservoirs de stockage afin de couvrir le pays en matière de carburants en parcourant une distance minimale.
La chaîne logistique est l'un des aspects les plus concernés, les plus améliorés et optimisés grâce à l'utilisation des méthodes et techniques d'optimisation de la recherche opérationnelle, comme les modèles mathématiques pour la planification, la distribution et la prévision .
Le problème que nous traitons a été posé par l'entreprise NAFTAL, plus précisément au niveau de la direction exploitation. Il s'agit d'optimiser la chaîne logistique ,notre mission consiste à trouver un schéma de distribution amélioré,réduire les coûts et répondre aux questions suivantes : **Quel est le meilleur emplacement des bacs de stockage pour maximiser la couverture du marché national ? Comment peut-on réduire les coûts de distribution tout en répondant aux besoins des stations services avec le plus rapide service de livraison ?**
Le présent document se compose de :
Chapitre 1 : Le premier chapitre est consacré à une présentation de l'organisme de l'entreprise NAFTAL, ses activités, et son mode de fonctionnement. Ensuite, nous examinons la problématique choisie.
Chapitre 2 : Dans le deuxième chapitre, nous citons la partie théorique d'un problème d'optimisation multi-objecctif pour acquérir une compréhension fondamentale.
Chapitre 3 : Dans ce chapitre, nous concentrons sur notre problème d'emplacement des reservoires de stockage. Il décrit les objectifs orientant ce travail, les contraintes spécifiques auxquelles notre approche doit faire face et situe notre problème par rapport aux problèmes classiques de la recherche opérationnelle. Ensuite, nous résolvons le problème en utilisant le solveur CPLEX.

Chapitre 4 : Dans ce chapitre, nous présentons quelques méthodes de résolution pour les problèmes multiobjectif.
Chapitre 5 : Dans ce chapitre, nous proposons quelques algorithmes pour résoudre notre problème ,nous justifions également notre choix d'approche pour résoudre le problème, puis nous l'implémentons en utilisant le langage Python.

Chapitre 1

Présentation de l'entreprise

Chapitre 1

1.1 Introduction

L'énergie en Algérie occupe une place prédominante dans le secteur économique , grâce à sa richesse et sa diversité en ressources naturelles.
« Le Pétrole : L'or noir du XXe siècle » cette précieuse ressource qui a permis à l'Algérie d'occuper les premières places en production et exportation.
Depuis les années 1970, la consommation mondiale est en croissance continue ce qui a poussé l'industrie pétrolière algérienne à se développer.
La gestion de ces ressources est confiée à SONATRACH (Société Nationale de Transport et de la Commercialisation des Hydrocarbures) , une entreprise qui s'est structurée par plusieures filiales spécialisées telles que NAFTEC pour le raffinage et NAFTAL pour la commercialisation et la distribution.

1.2 Organisation générale de NAFTAL :

FIGURE 1.1 – Logo de l'entreprise NAFTAL.

NAFTAL a entamé divers ajustements organisationnels depuis ses débuts .
Pour rester compétitive à l'échelle mondiale, l'entreprise a jugé qu'il est essentiel d'adopter une structure organisationnelle.

1.2.1 Direction générale :

Responsable de la stratégie et des orientations générales, la cohérence globale, la gestion et la surveillance stratégique.

1.2.2 Structures fonctionnelles :

Elles se composent de 3 directions :
Directions Exécutives : Elles ont pour mission :
_ Anticiper les tendances.
_ Assister les structures opérationnelles.
Directions centrales :
_ Élaborer les politiques et les stratégies de la société.
_ Appuier les structures opérationnelles.
Directions de Soutien :
_ Assurer la gestion administrative du siège social de la la société.

Structures opérationnelles :

Elles se divisent en 3 branches :

1. Branche carburant ;
2. Branche commercialisation ;
3. Branche GPL

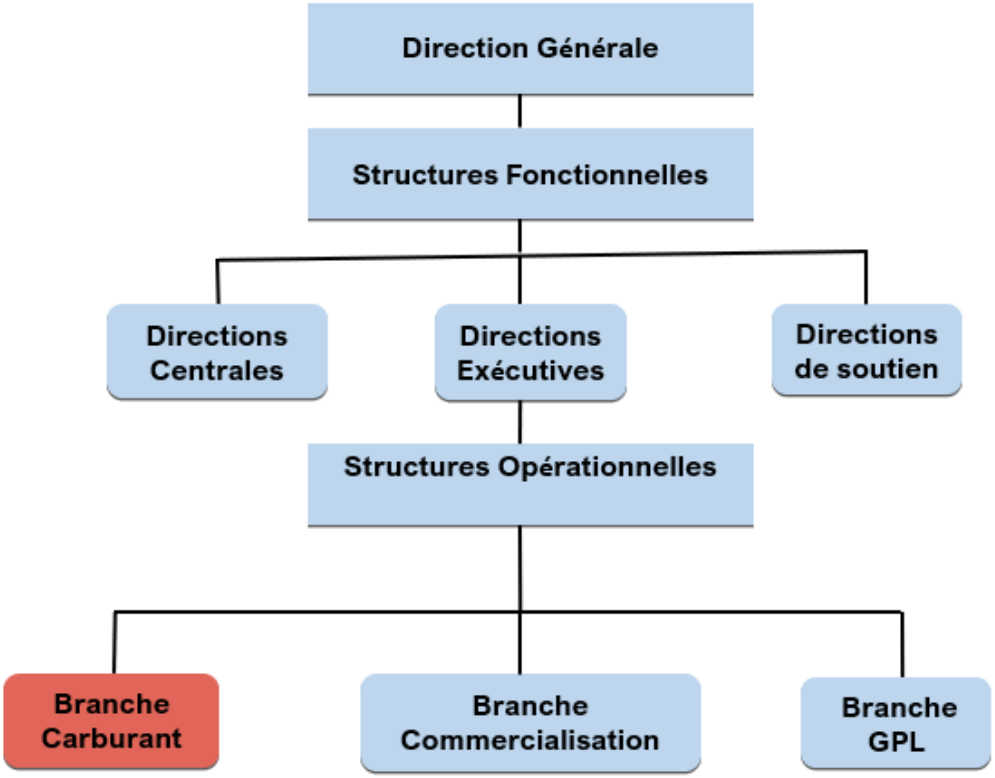


FIGURE 1.2 – Organisation générale du NAFTAL

1.3 Les activités du NAFTAL :

Elle a pour mission principale, la distribution et la commercialisation des produits pétroliers et dérivés sur le marché national.
Elle intervient également dans le domaine de :
• L'enfûtage des GPL(remplissage des fûts) ;
• La construction des bitumes ;
• La distribution, le stockage et la commercialisation des carburants, GPL, lubrifiants, bitumes, pneumatiques, GPL/carburant, produits spéciaux.

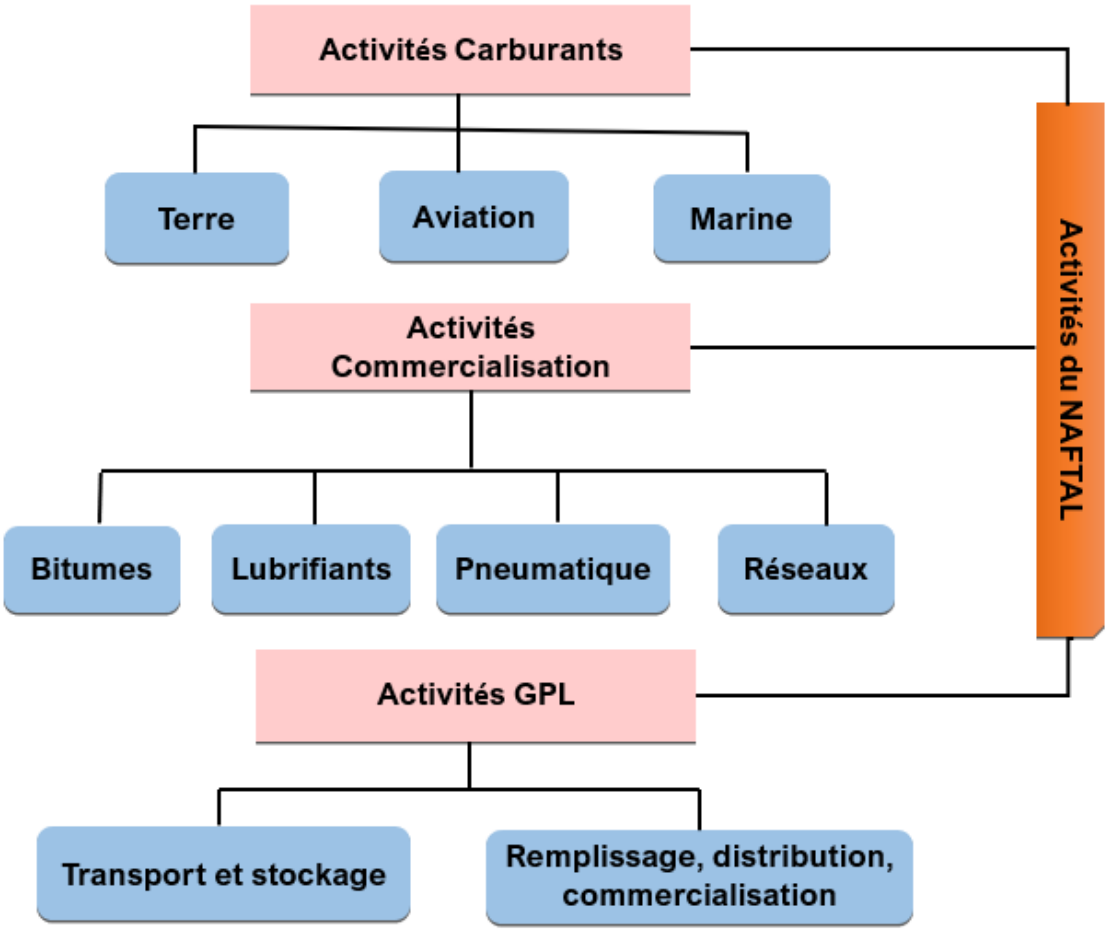


FIGURE 1.3 – Les activités du NAFTAL.

1.4 Les produits commercialisés par NAFTAL

1.4.1 Les carburants

L'essence sans plomb (ESP)

C'est un combustible issu du raffinage du pétrole et le plomb a été enlevé de celui-ci à cause de son impact sur l'environnement (la pollution) et sur la santé publique.

Le gasoil

C'est un carburant issu du raffinage du pétrole appelé aussi le diesel, il est composé principalement d'hydrocarbure plus lourds que l'esp.

1.4.2 Les lubrifiants (huiles et graisses)

Des matières onctueuses liquides ou solides, utilisées pour réduire les frottements.

1.4.3 Les Pneumatiques

Aussi appelés pneus, ce sont des composants qui entourent les roues des véhicules et assurent leur contact avec le sol.

1.4.4 Le bitume

Est dérivé du pétrole,il est visqueux et imperméable.

1.4.5 Le fuel (léger et lourd)

C'est une huile industrielle légère utilisée dans les chaudières.

1.5 Branche carburant

1.5.1 Localisation géographique

La branche carburant est située dans la wilaya d'Alger et plus précisément dans la circonscription administrative de DAR EL-BEIDA à 300 mètres de l'aéroport HOUARI BOU-MEDIENE.

1.5.2 Présentation de la branche carburant

La branche carburant de NAFTAL est la division responsable de la gestion des approvisionnements, du stockage et de la distribution des carburants pour l'aviation, la marine et les véhicules terrestres.
Elle contient deux divisions :
Division carburant terre :
chargée de distribuer :

- L'essence Sans Plomb ;
- Le Gasoil.

Division Aviation et Marine :

- Le Kérosène ;
- Le Fuel.

1.6 Les modes de transport

NAFTAL met en contribution 4 moyens de transport qui se distinguent en deux catégories :

1.6.1 Les moyens fixes

Transport par canalisation(pipe-line)

C´est un système industriel constitué des tubes qui contiennent des bouchons , des vannes , des robinets ...
Les pipes peuvent atteindre des centaines de kilomètres de longueur.
Le transport par pipe est moins couteux.

1.6.2 Les moyens mobiles

Transport par cabotage(navire)

Ce mode de transport concerne les entrepôts à proximité du littoral.
•Il est très sensible au dégradations climatiques.
•Ce moyen est utilisé pour approvisionner des entrepôts à partir la raffinerie de Skikda pour combler le déficit de production d'autres raffineries et couvrir les besoins en carburants de ces régions.

Transport par rail(wagon)

Il assure le transport des carburants des entrepôts vers les dépôts qui ont une capacité de stockage supérieure à $5000m^3$.

Transport par route(camion-citerne)

Il joue un rôle important dans la livraison des clients et le ravitaillement des dépôts non desservis par le rail.
• Pour le ravitaillement on utilise des camions mono-produit tandis que pour la livraison les camions sont multi-produits.
•Ce moyen de transport peut atteindre les zones d'accès les plus difficiles et les lieux les plus éloignés.

1.7 La gestion des stocks

Dans notre cas le stockage est une activité permanente , il y a toujours un flux entrant et un flux sortant, pour atteindre l´équilibre entre ces deux flux il nous faut un ensemble des mesures car l'espace de stockage est limité.
Cet ensemble s'appelle la gestion de stock , qui sert à identifier les produits existants et détecter les pénuries.

1.7.1 Les centres de stockage

Centres primaires(les entrepôts)

Ce sont des centres de distribution de grandes capacités de stockage des carburants ,ils s'approvisionnent à partir des centres de production (les raffineries) soit par canalisation soit par cabotage et ils approvisionnent les centres secondaires (dépôts) un entrepôt comprend :

1. Des postes de réception ;
2. Des réservoirs de stockage(des bacs) ;
3. Des postes de chargement ;
4. Un ensemble de canalisation et pompes qui transfèrent des produits entre les 3 groupes d'installations précédentes.

Chaque entrepôt couvre un ensemble de dépôts.

Centres secondaires(les dépôts)

Ce sont des centres de distribution dont les capacités de stockage sont inférieurs à celles des entrepôts.
Les dépôts sont ravitaillés par les entrepôts soit par wagon-citerne soit camion-citerne.
Les centres secondaires ont les mêmes composantes avec les centres primaires sauf qu'il y a des dépôts avec des cuves et d'autres avec des bacs et des cuves comme suit :

1.8 Le réseau de distribution

Le réseau de distribution peut être résumé en trois phases :

Bac	Cuve	Bac et cuve
Oued Aissi	Biskra	Laghouat
Batna	Khenchela	Dréa
Bechar	Ain Oussara	
M´sila	Djelfa	
Tebessa	Adrar	
El Eulma	Tindouf	
Ghardaia	BBA	
Touggourt	Jijel	
Bourached	Cheria	
Tiaret	Oued Djema	
	Djanet	
	El -Oued	
	In Salah	
	Tam	
	Aflou	
	Chlef	
	El-Bayad	
	Frenda	
	Tissemsilt	

TABLE 1.1 – Les réservoirs de stockage des centres secondaires

1.8.1 L'approvisionement

C'est L'acheminement des produits pétroliers **de la source (la raffinerie) vers un centre de stockage primaire (entrepôt)** .
Ce transfert est réalisé soit par canalisation (pipe -line) ou par cabotage (bateau).

1.8.2 Le ravitaillement

C'est le transfert des produits pétroliers **d'un centre primaire (entrepôt) vers un centre secondaire (dépôt)**.
Cette opération est réalisée soit par rail (wagon citerne) ou par route (camion).

1.8.3 La livraison

C'est la dernière phase qui a pour but de mettre le carburant à la disponibilité du consommateur, cette étape se fait par deux méthodes :
La livraison directe : Consiste à livrer les produits carburants **d'un dépôt vers les stations services**.
La livraison en droiture : Consiste à livrer les produits carburants **d'un entrepôt directement vers les CDD(les centres de distribution)**.

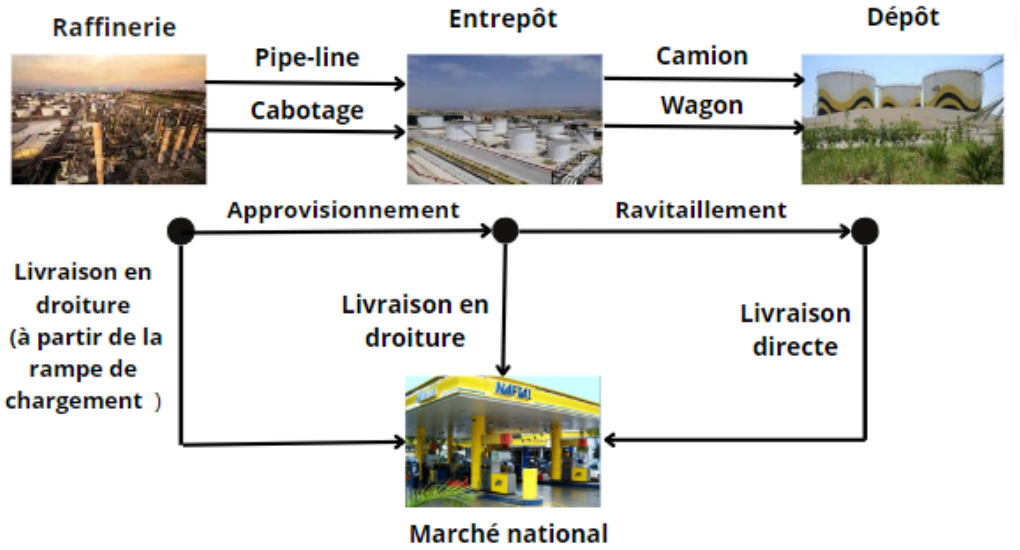


FIGURE 1.4 – Flux de distribution

1.9 Schéma de distribution actuel

Le flux de distribution se compose de 5 raffineries, 13 entrepôts et 31 dépôts qui sont dispatchés sur plusieurs régions comme suit :

1.9.1 La production des raffineries

Une raffinerie de pétrole est une usine où se déroule le processus de conversion du pétrole brut en produits finis(raffinage pétrolier) car le pétrole brut est inutilisable en l'état, ses composants doivent être séparés par distillation du plus leger(gaz et GPL) au plus lourd (GO) pour obtenir un produit final exploitable directement. Également connu sous le nom de source de pétrole ou de centre de production.

Raffinerie	Code
Skikda	RA1K
Alger	RA1G
Arzew	RA1Z
Hassi-Messaoud	RHMD
Adrar	RA1D

TABLE 1.2 – Liste des raffineries

Région EST

La région orientale est à la fois celle qui consomme le plus de carburant et qui en produit le plus grâce à la raffinerie de Skikda. Avec une production de 21 300 m³ /jr (1800 m³/jr en ESP et 19500 m³/jr en Go).
Cette région est la première. non seulement qu'elle couvre sa zone, elle couvre le manque de production d'autres zones, du centre et de l'ouest par cabotage et du sud par camion.

Région CENTRE

Est la deuxième région productrice et consommatrice de carburant, avec une production de 7000 m³/jr qui est insuffisante pour approvisionner ces entrepôts.

Région OUEST

La production de la région occidentale est de 5500 m3/jour, ce qui la place au troisième rang.

Région SUD

Cette zone est la plus grande en superficie et la plus petite en demande. Son schéma de distribution n'est pas assez simple à cause de plusieurs contraintes tel que :
.la difficulté d'atteindre cette région (trop loin)
.Les conditions météorologiques (tempêtes de sable) sont des obstacles pour le transport par wagon.
.La demande de gazoil est très élevée en été particulièrement à cause de l'utilisation des centrales électriques.
.Actuellement,il y a un manque de sécurité dans cette zone, des escortes interviennent pour prévenir toute tentative de vol.
.Le convoi est chargé pour transporter plusieurs produits, alors sont départ doit être planifié à une heure précise, pour cela l'entreprise doit assurer la présence du produit à temps.
.Chaque dépôt couvre son réseau (livraison directe)
Cette année NAFTAL préfère la livraison en droiture pour les stations services situés en amont des dépôts secondaires ou bien dans les régions qui ont seulement des entrepôts pour :
. Éviter le retour produit.
. Réduire le temps du livraison.

.Optimiser le schéma de distribution.
.Les opérateurs privés ont des accords similaires avec NAFTAL pour satisfaire leurs besoins en carburant à partir des dépôts de la société, en payant les services fournis.
Le shéma est représenté ci dessus :

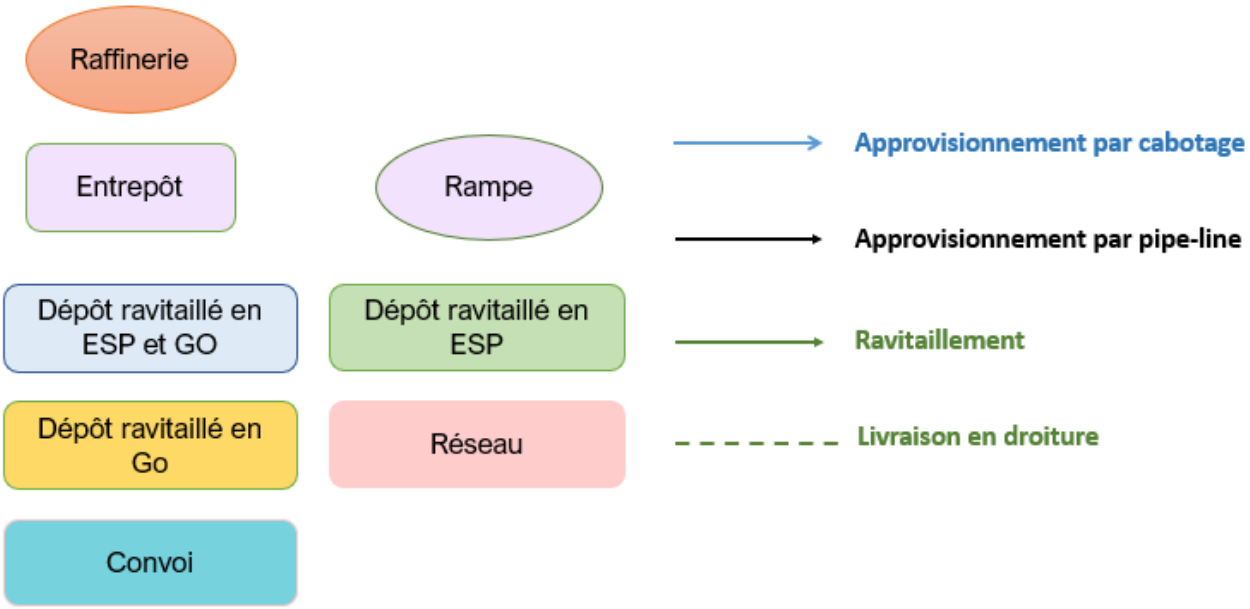


FIGURE 1.5 – Clé des Schémas

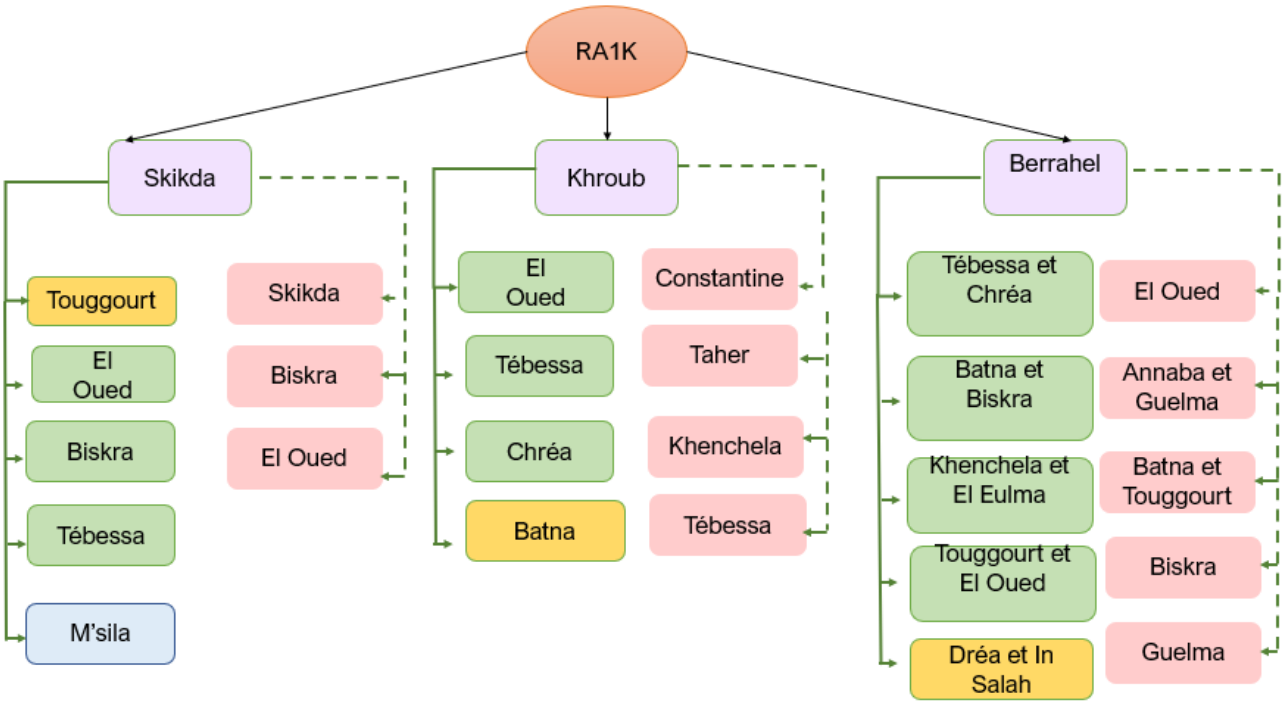


FIGURE 1.6 – Schéma de distribution de la région EST en démarrant par l'approvisionnement par pipe -line

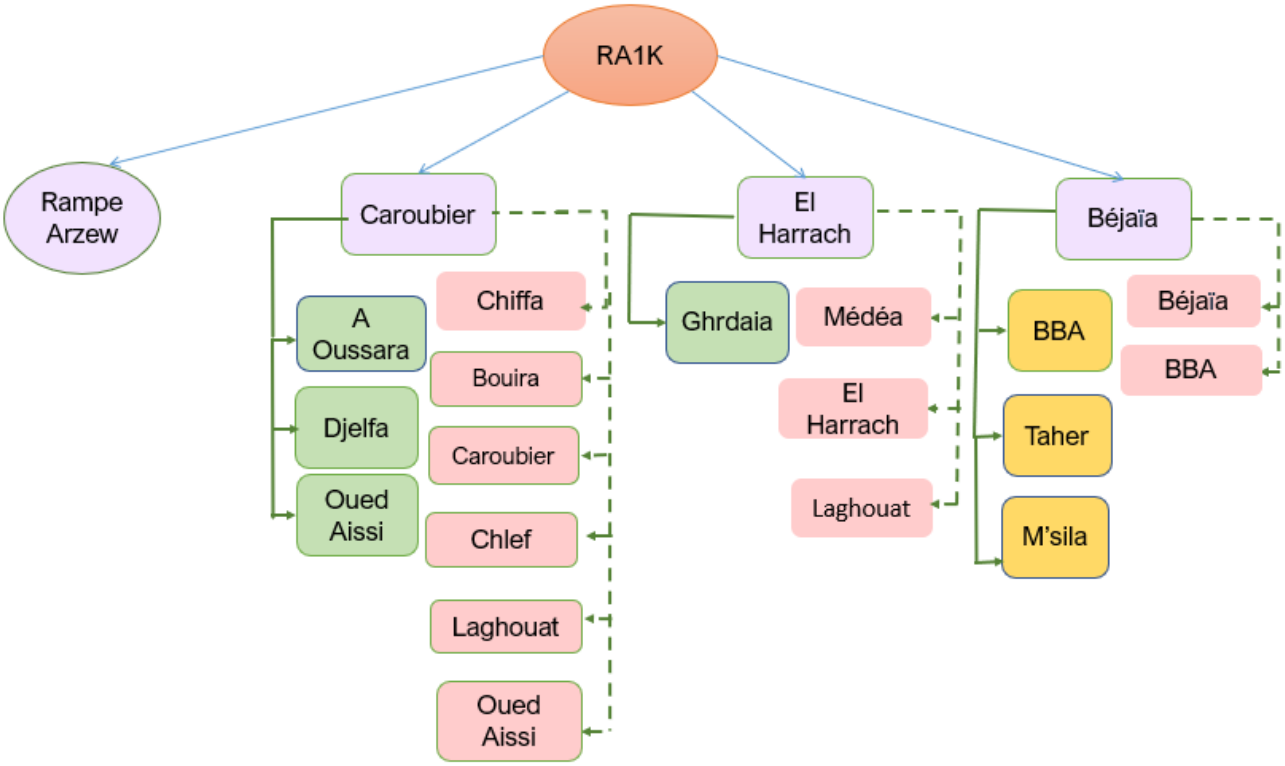


FIGURE 1.7 – Schéma de distribution de la région EST en démarrant par l'approvisionnement par cabotage

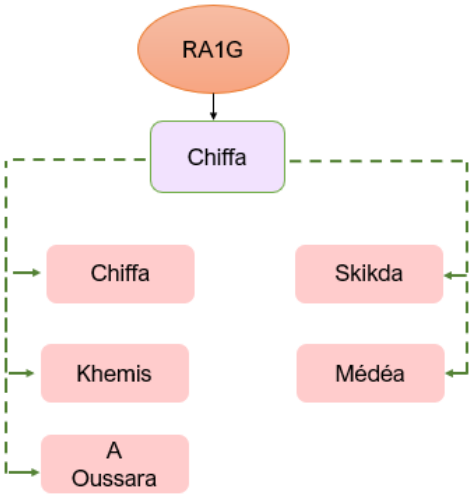


FIGURE 1.8 – Schéma de distribution de la région CENTRE

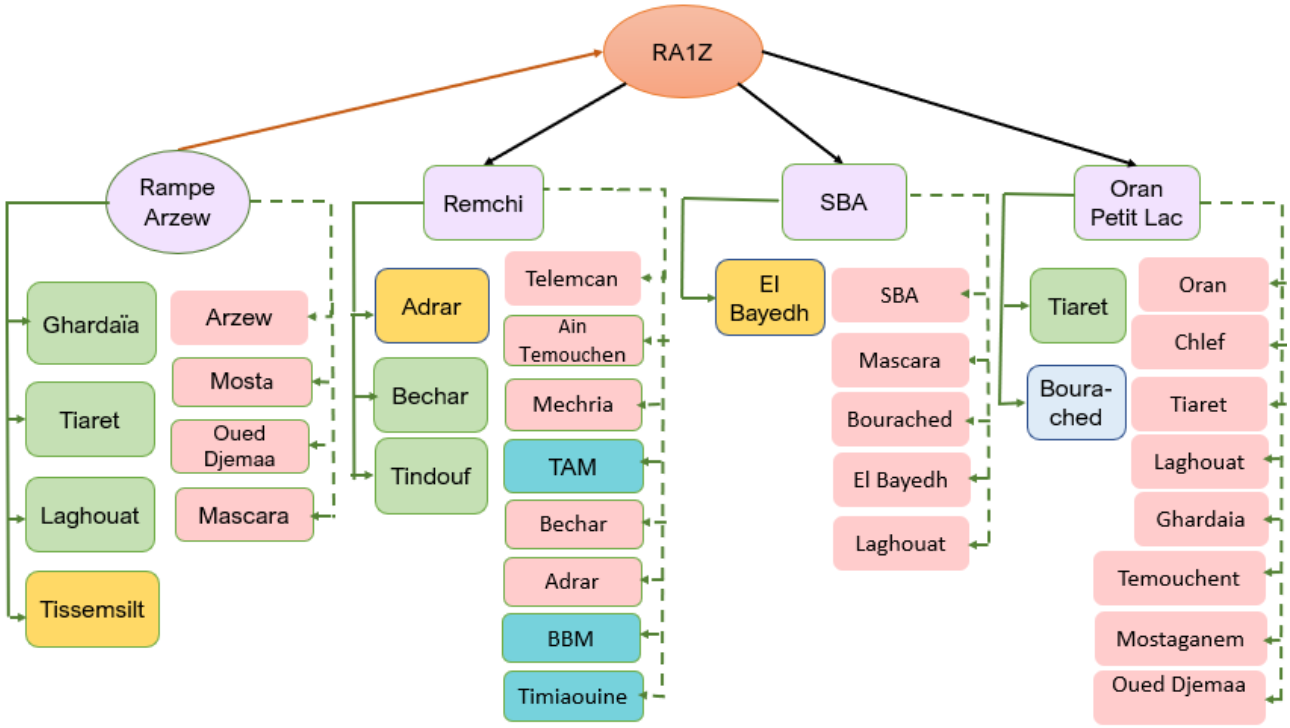


FIGURE 1.9 – Schéma de distribution de la région OUEST

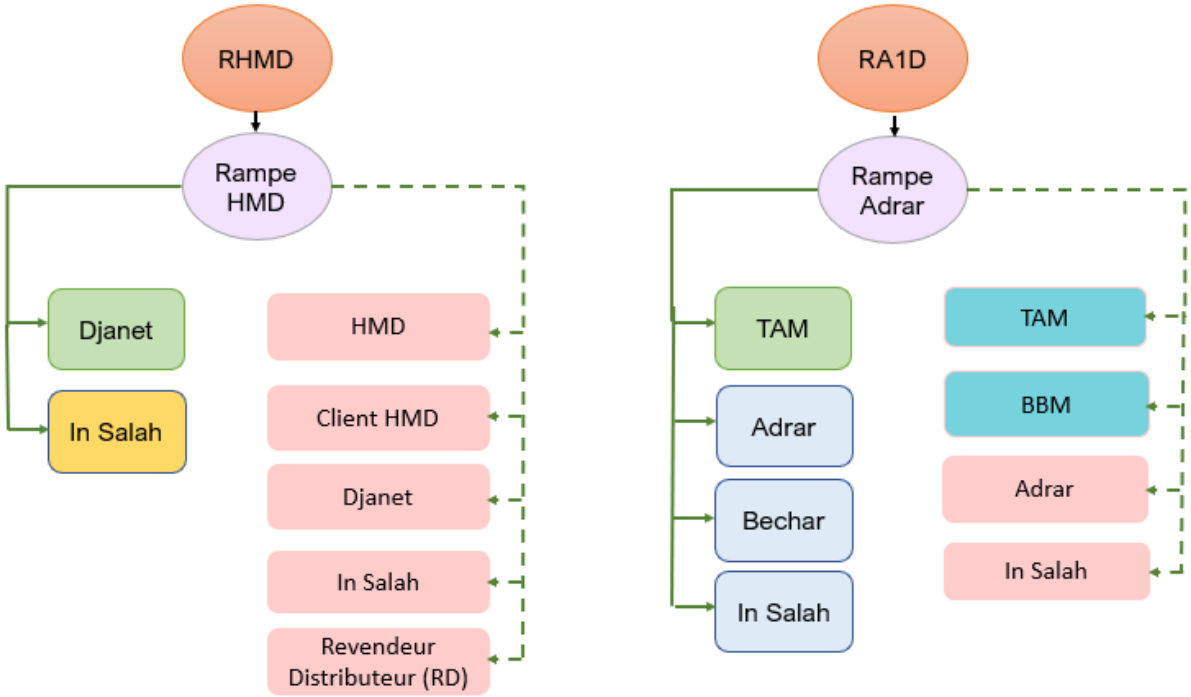


FIGURE 1.10 – Schéma de distribution de la région SUD

1.10 Problématique :

La branche carburant qui est responsable du stockage et de la distribution évite au mieux les ruptures de stock.
Elle doit assurer la couverture du marché dans le cadre d'amélioration du plan d'acheminement de ses produits, ceci nécessite une gestion efficiente de la distribution pour garantir la disponibilité des produits au bon moment et au bon endroit.
NAFTAL est en face d'une problématique purement technique qui est l'insuffisance de capacité de stockage ce qui a conduit à une insuffisance d'autonomie pour cela l'entreprise décide de construire des nouveaux entrepôts (des centres de stockage primaires) ou dépôts(des centres de stockage secondaires) de carburant terre et une nouvelle raffinerie à Hassi-Messoud afin d'assurer un meilleur schéma de distribution et de satisfaire la demande.

1.11 Les objectifs :

• Augmenter la capacité de stockage(intelligemment) dans le but d'atteindre 30 jours d'autonomie.
•Améliorer sa qualité de service.
• Réduire les coûts de transport.
•Optimiser le flux de distribution.
• Minimiser les dépenses du cabotage en construisant de nouveaux dépôts et une nouvelle
raffinerie.

1.12 Conclusion

Pour résumer, Ce chapitre renferme la macrostructure de NAFTAL, leur activités,leur produits stratégiques, leur modes de transport utilisés.
Ensuite, on a cité les différentes branches et en particulier la branche carburant là où on a réalisé notre stage et on a terminé par la présentation du schéma de distribution actuel et la problématique posée.
Les références [1],[13],[14]ont été utilisées dans la rédaction de ce chapitre.

Chapitre 2

Notions de base

Chapitre 2

L'optimisation combinatoire revêt une grande importance en recherche opérationnelle qui se justifie d'une part, par la difficulté des problèmes d'optimisation et d'autre part, par des nombreuses applications pratiques pouvant être formulées sous la forme d'un problème d'optimisation combinatoire. Bien que les problèmes d'optimisation combinatoire soient souvent faciles à définir, ils sont généralement difficiles à résoudre. En effet, la plupart de ces problèmes appartiennent à la classe des problèmes NP-difficile et ne possèdent donc pas à ce jour des solutions algorithmiques efficaces.

2.1 Définition du problème

Tout d'abord définissons trois notions de base d'un problème d'optimisation :
•**Fonction objectif :** La fonction objectif est la fonction à optimiser (maximiser ou mini- miser), appelée aussi fonction d'adaptation, fonction coût, fonction fitness, critère, utilité ou encore performance. On la note Z(x).
Elle attribue à chaque solution $x \in X$ de l'espace de recherche, un nombre réel indiquant sa valeur (son score).
• **Paramètre :** Le paramètre est une variable qui exprime une donnée quantitative ou qualitative au problème (exemple : coût, temps,. . .,etc).
Ce sont les variables de la fonction objectif, appelée aussi **variables de conception, variables d'optimisation ou encore variables de projet**.
• **Contrainte :** La contrainte est un ensemble des conditions qui doit être respectée par le vecteur de décision pour trouver la solution admissible.

2.2 Programme linéaire

La programmation linéaire est un outil générique qui peut résoudre un grand nombre de problèmes. On identifie dans la programmation linéaire, la

programmation linéaire en nombres réels, pour laquelle les variables des équations sont dans $\mathbb{R}_+$ et la programmation en nombres entiers, pour laquelle les variables sont dans $\mathbb{N}$

Un **PL** est un problème mathématique d'optimisation comportant des constantes, ou données et des variables. Ces derniers sont liées par des équations et / ou inégalités linéaires appelées **contraintes**

Un PL comporte en outre **une fonction objectif** linéaire pour les variables qu'il s'agit de maximiser ou minimiser.

Le modèle dit de forme générale se présente ainsi :

max ou min $(Z) = C_1X_1 + C_2X_2 + \cdots + C_rX_r + \cdots + C_nX_n$

$$\begin{cases} a_{11}X_1 + a_{12}X_2 + \cdots + a_{1r}X_r + \cdots + a_{1n}X_n \leq b_1 \\ \vdots \\ a_{s1}X_1 + a_{s2}X_2 + \cdots + a_{sr}X_r + \cdots + a_{sn}X_n \leq b_s \\ a_{s+1,1}X_1 + a_{s+1,2}X_2 + \cdots + a_{s+1,r}X_r + \cdots + a_{s+1,n}X_n \geq b_{s+1} \\ \vdots \\ a_{t1}X_1 + a_{t2}X_2 + \cdots + a_{tr}X_r + \cdots + a_{tn}X_n \geq b_t \\ a_{t+1,1}X_1 + a_{t+1,2}X_2 + \cdots + a_{t+1,r}X_r + \cdots + a_{t+1,n}X_n = b_{t+1} \\ \vdots \\ a_{m1}X_1 + a_{m2}X_2 + \cdots + a_{mr}X_r + \cdots + a_{mn}X_n = b_m \\ X_1, \ldots, X_n \geq 0 \end{cases}$$

Il renferme n variables $X_1, \ldots, X_n$,les données du problème sont les n nombres $C_1, \ldots, C_n$ les coéfficients de la fonction objectif,$b_1, \ldots, b_m$ les coéfficients à droite des contraintes , les $m \times n$ réels

on distingue trois parties toujours présentes :

— La fonction Z est appelée fonction objectif car elle traduit l'objectif du problème (par exemple minimiser la longueur de chutes de papiers lors d'une découpe, maximiser un profit ...)

— La dernière ligne indique que les variables doivent être non négatives.

— Les autres lignes concernent ce qu'on appelle les m contraintes du problème il y a inégalités du type $\leq$, t-s inégalités linéaires du type $\geq$ et m-t équations linéaires.

2.3 Problème d'optimisation

L'optimisation est la démarche fondée à rendre optimal le fonctionnement d'un système : Donner à quelque chose (par exemple à une machine, à une entreprise, etc.) le rendement optimal en créant les conditions les plus favorables
L'optimisation aide dans la prise de décision.

Définition 2.3.1. *Problème d'optimisation Soit $n \in N^*$ un entier strictement positif,*
$X \subset R^n$ un sous-ensemble non vide $f : X \to \mathbb{R}$ une fonction à valeurs réelles. Un problème d'optimisation consiste à trouver la meilleure solution $x \in X$appelée solution globale.

2.4 Solution réalisable et solution optimale

Soit P un un PL de la forme standard :

$$P \begin{cases} Max\, Z = c^t x \\ Ax = b \\ x \geq 0 \end{cases}$$

Définition 2.4.1. *- On appelle solution réalisable de P un vecteur $x \in R^n$ qui vérifie toutes les contraintes, c'est -à-dire x est tel que $Ax = b$ et $x \geq 0$.*
L'ensemble $R = \{x \in R^n | Ax = b, x \geq 0\}$ de toutes les solutions réalisables de P est appelé domaine réalisable de P.

Définition 2.4.2. *_ Un vecteur $x^* \in R^n$ est appelé solution optimale de P si c'est une solution réalisable (c'est -à-dire $Ax^* = b$ et x^*et ≥ 0 et si $c^*x^* \geq c^*x$ pour toute solution réalisable x .Autrement dit, une solution est optimale si c'est une solution réalisable qui parmi toutes les solutions réalisables, donne la plus grande valeur à la fonction objectif .D'une autre façon encore, une solution est optimale si on ne peut trouver une solution réalisable qui la domine.*

2.5 Problème d'optimisation combinatoire

Traditionnellement le mot **combinatoire** désigne le dénombrement de combinaisons d'éléments en relation avec le calcul de probabilités.
Un problème d'optimisation consiste à rechercher la valeur maximale ou minimale, appelée optimum global, d'une fonction $f : X \rightarrow R$. Comme maximiser une fonction f est équivalent à minimiser la fonction $-f$, nous considérons dans le reste de ce travail que les fonctions doivent être minimisées.

2.6 Les classes de complexité

2.6.1 La classe P :

Un problème de décision(ie les problèmes dont la réponse est oui ou non) est dit de complexité P ou de temps polynomial lorsque le nombre d'étapes nécéssaires pour trouver la solution optimale est limité par un polynôme et il existe au moins un algorithme pour le résoudre.

2.6.2 La classe NP(Non-déterministe Polynomial) :

Un problème de décision est NP si solution peut être déterminée et évaluée en un temps polynomial par une machine de Turing non déterministe.

2.6.3 La classe NP-Complet :

.Parmi les problèmes de la classe NP il y a un sous ensemble qui regroupe les problèmes les plus difficiles connus sous le nom **NP-complet**.
.Un problème NP-complet possède la propriété que tout problème dans NP peut être réduit en celui -ci dans un temps polynomial

Rappel

Une réduction est une façon de convertir un problème en un autre problème de telle sorte que, si le second problème est résolu, il peut être utilisé pour résoudre le premier problème. Supposons, par exemple, que vous souhaitiez vous orienter dans une nouvelle ville. Vous savez que ce serait facile si vous aviez une carte. Ceci démontre **la réductibilité**. Le problème de trouver son chemin dans la ville est réductible au problème de l'obtention d'une carte de la ville (Sipser 1996). **Réduction polynomiale** Un problème X se réduit polynomialement à un problème Y si et seulement si des instances du problème X peuvent être résolues avec un algorithme polynomial qui résolve le problème Y.

2.6.4 La classe NP-difficile :

Un problème est Np-difficile s'il est **plus difficile** qu'un problème Np-complet.

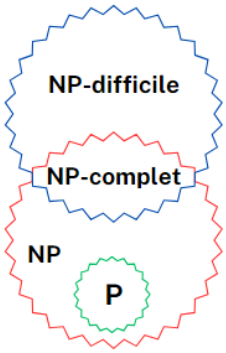


FIGURE 2.1 – La relation entre les classes du problème

2.7 Quelques problèmes d'optimisation combinatoire

L'optimisation combinatoire est un domaine vaste qui englobe une multitude de problèmes voici quelques problèmes célèbres :
•**Problème de sac à dos** Il s'agit de séléctionner les meilleurs objets possibles à mettre dans un sac à dos d'une capacité limitée
• **Problème de voyageur de commerce** Il s'agit de trouver le chemin le plus court reliant n villes données, chaque ville ne devant être visitée qu'une et une seule fois, et revenir à la ville de départ.

•**Problème de transport** Consiste à acheminer, à coût moins chèr, des marchandises depuis m origines (centres de production, usines, etc.) vers n destinations (clients, marchés, dépôts,. . ..).
•**Problème de localisation** De nombreux problèmes de décisions économiques consistent à sélectionner et/ou de placer certaines installations afin de répondre efficacement à des demandes données.
On peut par exemple s'intéresser au placement d'usines, d'installations de stockage, de dépôts, d'entrepôts, de casernes de pompiers etc. Ces problèmes ont en commun le fait que l'on doit choisir un ensemble d'installations, chacune ayant une certaine position, avec pour objectif de satisfaire au mieux les demandes (des clients, des utilisateurs, etc). Les problèmes de localisation, qui apparaissent également dans des contextes moins évidents, ont de nombreuses applications.

2.8 Optimisation mono ou multi-objectif

Les problèmes d'optimisation peuvent viser un ou plusieurs objectifs.
Un problème à objectif unique est défini par une seule fonction objectif appellé **mono-objectif**, s'il se trouve plusieurs objectifs potentiellement contradictoires alors on parle d'un problème **multi-objectif**.

2.9 Formulation générale d'un Problème multi-objectif

Un problème d'optimisation multiobjectifs(multicritères) consiste à optimiser(minimiser ou maximiser) k fonctions objectifs simultanément $(k \geq 2)$. Il est définit sous la forme suivante :
(PMO)

$$\begin{cases} \text{optimiser } Z(x) = (Z_1(x), Z_2(x), \ldots, Z_k(x)) \\ \text{s.c. } x \in S \end{cases}$$

Où :
– $x = (x_1, x_2, x_3, ...x_n)^t \in R^n$ représente le vecteur de décision avec $i = \overline{1,n}$
– R^n s'appelle espace de décision.
– $Z(x) = (Z_1(x), Z_2(x), ..., Z_k(x)) \in R^K$ représente le vecteur des fonctions objectifs avec Z_i les fonctions objectifs $(k \geq 2)$.
– R^k s'appelle espace des critères ou espace des objectifs
Avec :

$$Z : R^n \to R^k$$

$$x \rightarrow Z(x) = (Z_1(x), Z_2(x), ..., Z_k(x))$$

Et :

$$Z_i : R^n \rightarrow R$$

$$x \rightarrow Z_i(x),\ i = \overline{i,k}.$$

– S est l'ensemble des solutions réalisables,$S \subset R^n$.
– D = Z(S) est l'image de S par Z, il s'appelle espace des critères réalisables $D \subset R^k$, il est donnée par :

$$D = Z(S) = \{z \in R^k / z = Z(x); x \in S\}$$

Où z est le vecteur de l'espace des critères qui correspond à la solution réalisable x. pour cela nous avons présenté l'espace de décision et l'espace des critères dans la figure. dans notre PERS k=2 alors le problème est dit bi-objectif .

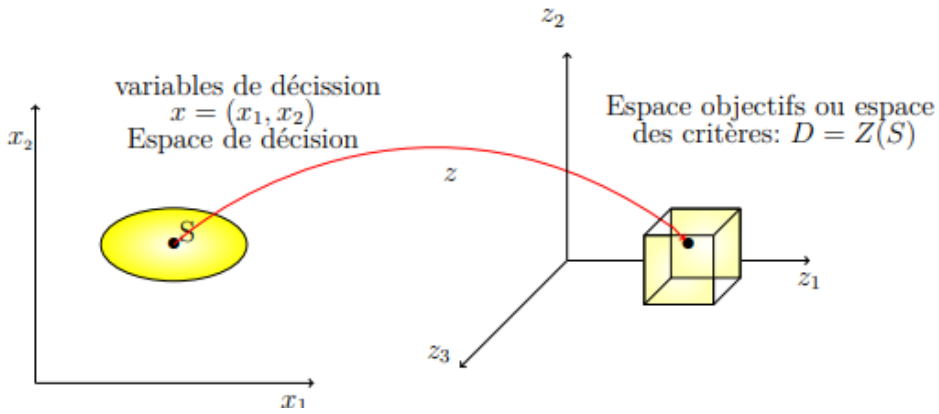


FIGURE 2.2 – Représentation de l'espace des décisions et l'espaces des objectifs correspondant.

2.10 La multiplicité des solutions

Lorsque l'on cherche à obtenir une solution optimale à un problème d'optimisation multiobjectif donné, on s'attend souvent à trouver une solution et une seule. En fait, il n'y a aucune combinaison des variables de décisions qui minimise (ou maximise) tous les composantes du vecteur Z simultanément.
Les POM ont un ensemble de solutions optimales dont les valeurs des fonctions objectifs sont les meilleurs compromis possibles dans l'espace des critéres.

2.11 Propriétés

Nous prèsentons ici les propriètès lièes au contexte multi-objectif.

2.11.1 Efiicacité et dominance

2.11.2 Solution efficace

Un point $x^* \in X$ est efficace s'il n'existe pas un autre point $x \in X$ tel que $F(x) \leq F(x^*)$ avec au moins une $F_i(x) < F_i(x^*)$. Sinon, x^* est inefficace.

2.11.3 La relation de dominance

Un vecteur de fonctions objectif $F(x^*) \in Z$ est non dominé s'il n'existe pas un autre vecteur $F(x) \in Z$ tel que $F(x) \leq F(x^*)$ avec au moins une $F_i(x) < F_i(x^*)$. Sinon, $F(x^*)$ est dominé.

2.11.4 Les points caractéristiques

Dans le cadre de l'optimisation multiobjectif le décideur raisonne de l'évaluation des solutions sur chaque critère et ce place naturellement sur l'espace des critères. Dans cette section nous allons définir le point idéal, nadir et la matrice des gains.
Point idéal Le vecteur idéal $y^* = (y_1^*, y_2^*, \ldots, y_n^*)$est obtenu en optimisant séparément chaque fonction objectif f_i ie $y^* = f_i^*(x)$, $x \in S$
. ses coordonnées correspondent aux valeurs favorables de chaque f_j
.Le décideur peut le définir comme un point de référence exprimant le but qu'il veut atteindre pour chaque critère.
Point nadir Est défini dans le cas de minimisation par :

$$N^d = \{\max_{\forall x \in Deff} Z_1(x), \max_{\forall x \in Deff} Z_2(x), \ldots, \max_{\forall x \in Deff} Z_n(x)\} \in \mathbb{R}^n$$

où Deff est l'ensemble des solutions efficaces.

$$N^d = \left(\max_{\forall y \in Y_N} y_1, \max_{\forall y \in Y_N} y_2, \ldots, \max_{\forall y \in Y_N} y_n\right)$$

où y_n est l'ensemble des solutions non dominés

.ses coordonnées correspondent aux pires valeurs obtenues par chaque fonction objectif sur l'ensemble de solutions non dominés .
Matrice des gains La matrice carrée de dimension n suivante

$$\begin{bmatrix} Z_{11} & Z_{12} & \dots & Z_{1n} \\ Z_{21} & Z_{22} & \dots & Z_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ Z_{n1} & Z_{n2} & \dots & Z_{nn} \end{bmatrix}$$

est la matrice des gains ou pay off table notée G , où $Z_{ij} = Zi(x_j)$,$\forall i, j = 1, \dots, n$. Les coordonnées du point idéal apparaissent sur la diagonale prinicipale de cette matrice

Les sources consultées pour ce chapitre sont :[2],[3],[4],[6],[7],[8],[9],[16],[17]

Chapitre 3

Modélisation

Chapitre 3

3.1 Introduction

Dans ce chapitre on s'intéresse aux deux premiers objectifs cités précédemment en s'appuyant sur l'optimisation bi-objectif.
On met la lumière sur la modélisation du notre problème pour simplifier la situation existante ,évaluer les performances ,analyser et permettre aux décideurs de faire les choix les plus efficaces.

3.2 Exemple Introductif

Prenons l'entrepôt de SBA(Sidi Bel Abbas) comme exemple de capacité de stockage minimale, Dans le but d'accroître cette capacité, nous envisageons la construction de nouveaux bacs.
Comme le tableau ci-dessus le démontre :

Région	**Entrepôt**	**Stock en (m^3)**	**Réseau**	**Demande**	**Autonomie**	**Produit**
Ouest	SBA	4916	SBA	486	4	GO et ESP
			Mascara	297		GO et ESP
			Bourached	135		GO et ESP
			El Bayedh	270		GO et ESP
			Laghouat	81		GO

TABLE 3.1 – Exemple d'un entrepôt avec une faible autonomie

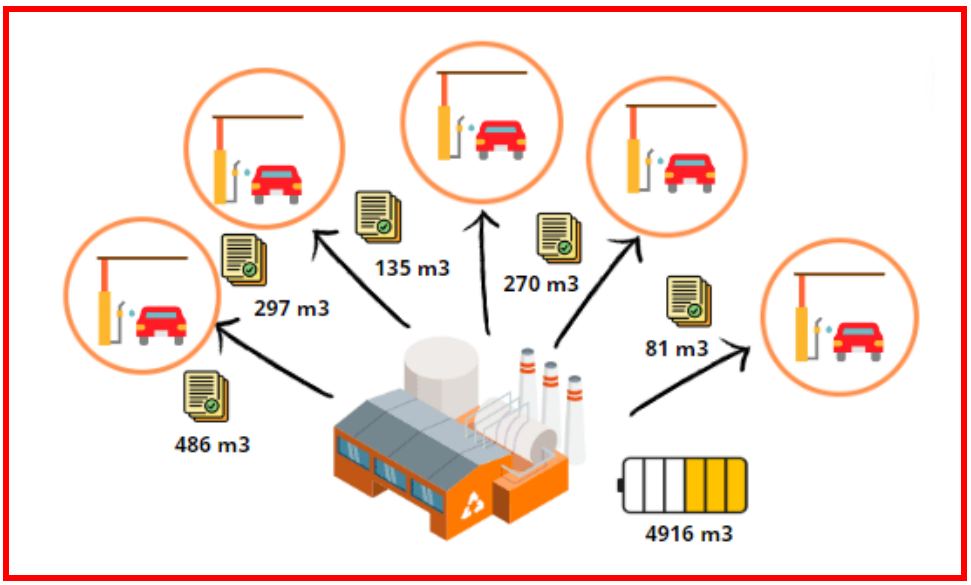


FIGURE 3.1 – Exemple sur la livraison en droiture (Entrepôt SBA -> Réseau)

D'après le schéma on remarque que la faible autonomie de cet entrepôt est due à la baisse du stock par rapport à la demande en raison d'un niveau de stockage insuffisant.
En cas d'une situation exceptionnelle, l'entrepôt tombe en rupture de stock.
La question crucial qui se pose est
" Combien de bac à installer pour améliorer l'autonomie en minimisant les coûts engendrés ? "

3.3 Position du problème

L'autonomie est la garantie que le pays peut maintenir son fonctionnement habituel en cas de perturbations sans nécessité d'un nouveau approvisionnement de carburant.

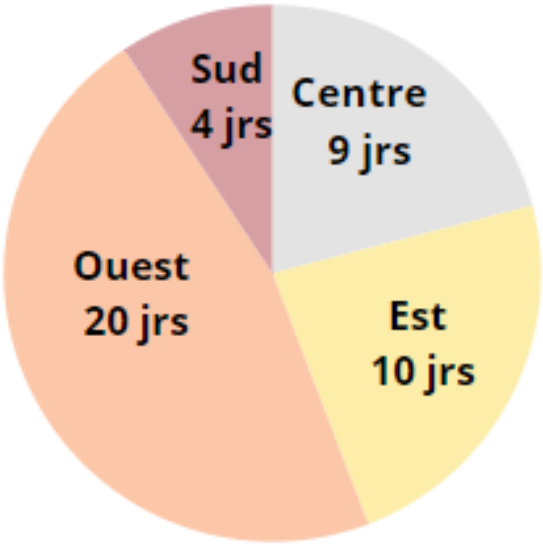


FIGURE 3.2 – Autonomie du GO sur le térritoire national

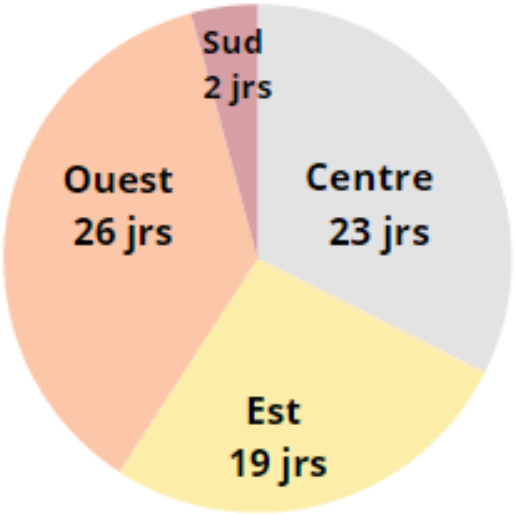


FIGURE 3.3 – Autonomie d'ESP sur le térritoire national

D'après le permier graphique nous pouvons noter que la région OUEST dispose de la plus grande autonomie en GO avec 20 jours, tandis que la région SUD a la plus faible autonomie avec seulement 4 jours. Les régions CENTRE et EST ont une autonomie proche, autour de 9-10 jours.
Dans ce second graphique, La région OUEST conserve la plus grande autonomie en ESP, atteignant cette fois-ci 26 jours, le CENTRE suit avec 23 jours d'autonomie.
L'EST affiche 19 jours d'autonomie.Cependant, la région SUD se démarque à nouveau avec seulement 2 jours d'autonomie, un niveau nettement inférieur aux autres zones.
Un écart aussi important dans l'autonomie des deux produits entre le nord et le sud.
NAFTAL a décidé de faire une extension d'entrepôts en priorisant le sud qui entraînera des coûts considérables parmi ces frais, on mentionne :

Les coûts d'investissement :

1. Construction
2. Aménagement et équipement (rayonnages, systèmes de manutention, ….)
3. Infrastructures (routes, parkings, réseaux, ….)

Les coûts d'exploitation :

1. Énergie (chauffage, climatisation, éclairage,. ….)
2. Main-d'œuvre (personnel de gestion, de manutention, de maintenance, ….)
3. Entretien et maintenance des équipements
   Assurances et taxes foncières.
4. Logiciels et systèmes informatiques.
5. Coûts de stockage (en fonction des volumes, types de produits, ….)

Autres coûts potentiels :

1. Sécurité et surveillance
2. Coûts environnementaux (conformité, traitement des déchets, ….)

Et c'est là que nous pouvons apporter notre contribution en tant que chercheur opérationnel en utilisant les connaissances acquises durant notre cursus universitaire, nous avons suggéré de ne pas construire de nouveaux entrepôts en raison des dépenses, de se limiter aux entrepôts existants et d'agrandir leur capacité en installant de nouveaux bacs de différents capacités pour optimiser le stock au même temps nous essayerons de faire un nouveau schéma de distribution pour

optimiser le transport (pour maîtriser les coûts des convois du nord vers le sud par exemple de l'entrepôt REMCHI vers TIMIAOUINE).
Suite à la prolongation de la durée d'auto-suffisance qui est le stock sur la demande, nous allons traiter **Le problème d'emplacement des réservoirs de stockage** qui consiste à déterminer le meilleur système d'implantation de ces derniers.
Il peut être formulé sous la forme d'un modèle de programmation linéaire en nombre réel pour ce faire, des variables de décision réelles et entières interviennent tout en respectant les contraintes fixées par le décideur selon ses besoins et ses préférences, en installant des bacs si c'est utile afin de maximiser le stockage des entrepôts et minimiser les coûts associés.

3.4 Caractéristiques du PERS

1. On prend en charge la commercialisation **des carburants terre (gasoil et l'essence sans plomb)**.
2. Les camions de la livraison sont conçus pour transporter les deux produits à la fois.
3. Grâce à la grande importance des entrepôts,on se base seulement sur l'amélioration de leurs capacités.
4. Les dépôts seront considérés comme des stocks de sécurité.
5. Les entrepôts sont disponibles tous les jours 24 heures sur 24.
6. Chaque résevoir de stockage contient un produit (deux produits ne peuvent pas être mélangés).

3.5 La formulation mathématique

Avant de formuler le problème nous introduisons les paramètres suivants :
Les données
R : Ensemble des raffineries .$(r \in R)$
I : Ensemble des entrepôts ,$(i \in I)$
J : Ensemble des types de bacs, $(j \in J)$
P : Ensemble des produits, $(p \in P)$
K : Ensemble des stations-services
CP_{rp} : Capacité de production de la raffinerie r du produit p.
CE_{ip} : Capacité actuelle de stockage de l'entrepôt i en produit p, $i \in I$, $p \in P$

CB_j : Capacité de stockage d'un bac de type j, $j \in J$
d_{ik} : la distance entre l'entrepôt i et la station service k
$A = ((a_{ik}))_{(i \in I, k \in K)}$ telle que

$$a_{ik} = \begin{cases} 1 & \text{si l'entrepôt est au voisinage de la station service} \\ 0 & \text{sinon} \end{cases} \quad , \forall i \in I, \forall k \in K$$

D_{kp} : Demande mensuelle estimée de la station-service k en produit p
B : Budget de l'entreprise NAFTAL spécialisé pour la construction
f_j : Coût d'installation d'un bac de type j , $j \in J$
s_p : Coût unitaire de stockage du produit p, $p \in P$
c_{ik} : Coût de transport unitaire, entre l'entrepôt i et la station-service k, $i \in I$, $k \in K$
c_{ri} : Coût de transport unitaire, entre la raffinerie r et l'entrepôt i.

Les Hypothèses

H1 :la demande est stable.
H2 :les réservoirs de stockage permettent d'augmenter la capacité à moindre coût par rapport à la construction de nouveaux centres primaires.
H3 :certains dépôts primaires n'ont pas besoin d'avoir une extension supplémentaire.
H4 :On suppose que la raffinerie de Hassi Messoud est mise en œuvre avec une capacité de production de 4 millions de tonnes.

Les variables de décision

x_{ijp} : nombre de bacs de type j à installer à l'entrepôt i pour stocker le produit p
y_{rip} : La quantité du produit p transportée à partir de la raffinerie r vers l'entrepôt i
z_{ikp} : La quantité livrée du produit p à partir de l'entrepôt i vers la station service k

Les contraintes :

1. La quantité transportée de la raffinerie r vers l'entrepôt i du produit p doit être au plus égale à la capacité de production de la raffinerie :

$$\sum_{i \in I} y_{rip} \leq CP_{rp}; \quad \forall r \in R, \forall p \in P \tag{3.1}$$

2. La disponibilité du produit p dans l'entrepôt i doit être au moins égale à la demande de l'ensemble des stations-services k couvertes par cet entrepôt :

$$\mathrm{CE}_{ip} + \sum_{j \in J} \mathrm{CB}_j x_{ijp} \geq \sum_{k \in K} a_{ik} z_{ikp}; \quad \forall i \in I, \forall p \in P \tag{3.2}$$

La différence représente la quantité stockée du produit p dans l'entrepôt i à la fin du mois.

3.les bacs installés doivent être remplis :

$$\sum_{j\in J} CB_j x_{ijp} \leq \sum_{r\in R} y_{rip} \quad \forall i \in I, \forall p \in P \tag{3.3}$$

4. la demande mensuelle de chaque station-service k en produit p doit être au moins satisfaite :

$$\sum_{i\in I} a_{ik} z_{ikp} \geq D_{kp}; \quad \forall k \in K, \forall p \in P \tag{3.4}$$

5. Le coûts d'installation des bacs dans l'ensemble des entrepôts ne doit pas dépasser le budget d'invesstissement :

$$\sum_{i\in I}\sum_{j\in J}\sum_{p\in P} f_j x_{ijp} \leq B \tag{3.5}$$

6.les contraintes de positivité
$x_{ijp} \geq 0$, entiers,$\forall i \in I, \forall j \in J, \forall p$
$z_{ikp} \geq 0$,$\forall i \in I, \forall k \in K, p \in P$

La fonction objectif

Minimiser les coûts d'installation des bacs, du stockage des produits et du transport des produits :

$$\min Z_1 = \sum_{i\in I}\sum_{j\in J}\sum_{p\in P} f_j x_{ijp} + \sum_{p\in P} s_p \sum_{i\in I}\left(\left(\mathrm{CE}_{ip} + \sum_{j\in J}\mathrm{CB}_j x_{ijp}\right) - \sum_{k\in K} a_{ik} z_{ikp}\right)$$

$+ \sum_{i\in I}\sum_{k\in K}\sum_{p\in P} c_{ik} z_{ikp} + \sum_{r\in R}\sum_{i\in I}\sum_{p\in P} y_{rip} c_{ri}$

Maximiser le nombre de bacs à installer :

$$\max Z_2 = \sum_{i\in I}\sum_{j\in J}\sum_{p\in P} CB_j x_{ijp}$$

3.6 Modèle mathématique

$$
\begin{cases}
\min Z_1 = \sum_{i\in I}\sum_{j\in J}\sum_{p\in P} f_j x_{ijp} + \sum_{p\in P} s_p \sum_{i\in I}\left(\left(\mathrm{CE}_{ip} + \sum_{j\in J}\mathrm{CB}_j x_{ijp}\right) - \sum_{k\in K} a_{ik} z_{ikp}\right) \\
+\sum_{i\in I}\sum_{k\in K}\sum_{p\in P} c_{ik} d_{ik} z_{ikp} + \sum_{r\in R}\sum_{i\in I}\sum_{p\in P} y_{rip} c_{ri} \\
\max Z_2 = \sum_{i\in I1}\sum_{j\in J}\sum_{p\in P} CB_j x_{ijp} \\
\sum_{i\in I} y_{rip} \leq CP_{rp}, \quad \forall r\in R, \forall p\in P \\
\mathrm{CE}_{ip} + \sum_{j\in J}\mathrm{CB}_j x_{ijp} \geq \sum_{k\in K} a_{ik} z_{ikp}, \quad \forall i\in I, \forall p\in P \\
\sum_{j\in J} CB_j x_{ijp} \leq \sum_{r\in R} y_{rip}, \forall i\in I, \forall p\in P \\
\sum_{i\in I} a_{ik} z_{ikp} \geq D_{kp}, \quad \forall k\in K, \forall p\in P \\
\sum_{i\in I}\sum_{j\in J}\sum_{p\in P} f_j x_{ijp} \leq B, \\
\mathrm{x}_{ijp} \geq 0, \quad \text{entiers}, \forall i\in I, \forall j\in J, \forall p\in P \\
\mathrm{z}_{ikp} \geq 0, \quad \forall i\in I, \forall k\in K, \forall p\in P \\
\mathrm{y}_{rip} \geq 0, \quad \forall r\in R, \forall i\in I, \forall p\in P
\end{cases}
$$

3.7 Évaluation du modèle

(P) est un programme linéaire en nombres entiers dont la taille est :
Nombre de variables :
pour les

$$x_{ijp} = |I| \times |J| \times |P| = 13 \times 4 \times 2 = 104$$
$$y_{rip} = |R| \times |I| \times |P| = 5 \times 13 \times 2 = 130$$

$$z_{ikp} = |I| \times |K| \times |P| = 13 \times 629 \times 2 = 16354$$

Au total on a 16 588 variables Nombre de contraintes :

$$NC = |R| \times |P| + |I| \times |P| + |I| \times |P| + |K| \times |P| + 1$$
$$NC = |P|(2 \times |I| + |K| + |R||) + 1$$
$$NC = 2 \times (2 \times +629 + 5) + 1 = 1320$$

Au total on a 1320 contraintes

Pour une résolution d'un problème économique , le modèle ne peut jamais refléter la situation réelle avec fiabilité totale .

Remarque :

> Les coûts de stockage engendrés par les rampes de chargement sont négligeables.Ils sont supportés par SONATRACH.
> Dans le sud les coûts de transport sont nulles car les rampes (les entrepôts fictifs) se situent dans les raffineries .

3.8 Validation du modèle

CPLEX est une hybridation de deux éléments clés :

1. **L'algorithme du Simplexe :** CPLEX s'appuie sur les principes de l'algorithme du simplexe.
   Cet algorithme consiste à passer d'une solution de base rélisable à une autre donnant une meilleure valeur à la fonction objectif ;géométriquement ,passer successivement d'un sommet du Domaine S à un autre sommet de ce dernier tout en améliorant à chaque fois la valeur de la fonction objectif
2. **Le langage de programmation C :** Le nom "CPLEX" fait référence à **"C Programming Language Extension"**, ce qui indique que le solveur a été conçu pour s'intégrer étroitement au langage C.
   Cette connexion étroite avec le C permet une programmation flexible et une intégration facile dans divers environnements de développement.
   CPLEX optimizer
   Pour tester le modèle nous avons fait reccours au IBM CPLEX Optimizer un solveur reconnu pour traiter des problèmes NP-difficiles.

OPL(Optimization Programming Language)est un vocabulaire(un langage,une grammaire) permettant de décrire ces problèmes d'optimisation dans une syntaxe que les solveurs peuvent exploiter.
L'interface utilisée pour cela est **IBM Ilog Cplex Optimisation Studio**

FIGURE 3.4 – Logo Cplex Optimizer

Pour pouvoir valider notre modèle sur CPLEX nous avons dû procéder à une agrégation pondérée de nos fonctions objectifs car il ne prend pas en charge l'optimisation multi-objectif.
Pour combiner un problème de minimisation et un problème de maximisation dans une agrégation linéaire certe qu'il faut les convertir pour être sur la même base (soit tous des problèmes de maximisation , soit tous des problèmes de minimisation) pour rendre la combinaison des objectifs cohérente et interprétable.
1.**Transformation en un problème de minimisation**

$$
\begin{cases}
\min Z_1 = \sum_{i\in I}\sum_{j\in J}\sum_{p\in P} f_j x_{ijp} + \sum_{p\in P} s_p \sum_{i\in I}\left(\left(\mathrm{CE}_{ip} + \sum_{j\in J}\mathrm{CB}_j x_{ijp}\right) - \sum_{k\in K} a_{ik} z_{ikp}\right) \\
+\sum_{i\in I}\sum_{k\in K}\sum_{p\in P} c_{ik} d_{ik} z_{ikp} + \sum_{r\in R}\sum_{i\in I}\sum_{p\in P} y_{rip} c_{ri} \\
\min Z_2 = -\sum_{i\in I1}\sum_{j\in J}\sum_{p\in P} CB_j x_{ijp} \\
\sum_{i\in I} y_{rip} \leq CP_{rp}; \quad \forall r\in R, \forall p\in P \\
\mathrm{CE}_{ip} + \sum_{j\in J}\mathrm{CB}_j x_{ijp} \geq \sum_{k\in K} a_{ik} z_{ikp}, \quad \forall i\in I, \forall p\in P \\
\sum_{j\in J} CB_j x_{ijp} \leq \sum_{r\in R} y_{rip}, \forall i\in I, \forall p\in P \\
\sum_{i\in I} a_{ik} z_{ikp} \geq D_{kp}, \quad \forall k\in K, \forall p\in P \\
\sum_{i\in I}\sum_{j\in J}\sum_{p\in P} f_j x_{ijp} \leq B, \\
\mathrm{x}_{ijp} \geq 0, \quad \text{entiers}, \forall i\in I, \forall j\in J, \forall p\in P \\
\mathrm{z}_{ikp} \geq 0, \quad \forall i\in I, \forall k\in K, \forall p\in P \\
\mathrm{y}_{rip} \geq 0, \quad \forall r\in R, \forall i\in I, \forall p\in P
\end{cases}
$$

2.**fixation des poids** w_1 **et** w_2 tel que $w_1 + w_2 = 1$ et $w_2 > w_1$
Le code est écrit de la manière suivante dans un fichier(.mod)

```
// Définition des paramètres
int CaP[R][P]=...;// Capacité de production des raffineries
int CE[I][P] = ...; // Capacité actuelle de stockage de l'entrepôt i en produit p
int CB[J] = ...; // Capacité de stockage d'un bac de type j
int a[I][K] = ...; // 1 si l'entrepôt est au voisinage de l'entrepôt et 0 sinon
float D[K][P] = ...; // Demande mensuelle estimée de la station-service k en produit p
int f[J] = ...; // Coût d'installation d'un bac de type j
float s[I][P] = ...; // Coût unitaire de stockage de l'entrepôt i du produit p
float c[I][K] = ...; // Coût de transport unitaire entre l'entrepôt i et la station-service k
float CR[R][I]=...; //Coût de transport unitaire entre la raffinerie et l'entrepôt
// Définition des variables de décision
dvar int+ x[I][J][P]; // Nombre de bacs de type j installés dans l'entrepôt i
dvar float+ y[R][I][P];//Quantité de produit p transportée de la raffinerie vers l'entrepôt
dvar float+ z[I][K][P]; // Quantité de produit p livrée par l'entrepôt i à la station-service k
```

```
// Définition de la fonction objectif
minimize
    w1 * (sum(i in I, j in J, p in P) f[j] * x[i][j][p]
    + sum( p in P) s[p] *sum(i in I) (CE[i][p] + sum(j in J) CB[j] * x[i][j][p])
    - sum(i in I, k in K, p in P) a[i][k] * z[i][k][p]
    + sum(i in I, k in K, p in P) c[i][k] * z[i][k][p] +sum (r in R,i in I , p in P)y[r][i][p] * CR[r][i])
    -w2 * sum(i in I, j in J, p in P) CB[j] * x[i][j][p];
```

```
// Définition des contraintes
subject to {
      //Disponibilité du produit dans les raffineries
      forall (r in R, p in P)
     sum(i in I) y[r][i][p] <= CaP[r][p];

     // Disponibilité du produit dans les entrepôts
     forall (i in I, p in P)
         sum(j in J) CE[i][p] + CB[j] * x[i][j][p] >= sum(k in K)  a[i][k] * z[i][k][p];
         forall (i in I, p in P)
        sum(j in J) CB[j] * x[i][j][p]<= sum(r in R)y[r][i][p];
     // Satisfaction de la demande des stations-services du sud
     forall (k in K, p in P)
         sum(i in I)  a[i][k] *z[i][k][p] >= D[k][p];

     // Contrainte de budget
     sum(i in I, j in J, p in P) f[j] * x[i][j][p] <= B;


     // Contrainte de non-négativité
     forall (i in I, j in J, p in P)
         x[i][j][p] >= 0;

         forall ( r in R , i in I, p in P)
         y[r][i][p] >= 0;

     forall (i in I, k in K, p in P)
         z[i][k][p] >= 0;
            }
```

Cette étape nous a permis de convertir notre problème bi-objectif en un problème mono-objectif solvable par CPLEX et nous a donné une première indication sur la capacité du modèle à produire des solutions réalisables sans identifier l'ensemble des compromis possibles entre les deux objectifs .

3.9 Conclusion

Ce chapitre est le noyau de notre étude nous avons détaillé notre problème d'emplacement des bacs de stockage ainsi que la procédure de modélisation. après nous avons présenté un modèle mathématique (un programme linéaire en nombre entier) du notre problème traité tout en précisant sa complexité.

Chapitre 4

Méthodes de résolution

Chapitre 4

4.1 Introduction

Après la réalisation du modèle mathématique nous entammons dans ce chapitre la résolution de notre problème. Par la suite nous allons choisir une méthode de résolution selon la qualité des solutions et le temps d'exécution en prenant en compte les caractéristiques du problèmatique posée.

4.2 Approche de résoultion des problèmes(PBO)

Les problèmes d'optimisation bi-iobjectif consistent à optimiser deux critères simultanément, ce qui conduit à rencontrer des difficultés pour définition la solution optimale .En fait,il n'éxiste pas une solution unique qui optimise les 2 objectifs en même temps. Le décideur se retrouve donc confronté à un ensemble de solutions parmi lesquelles il doit exprimer ses préférences.

4.3 Les méthodes exactes

Sont des algorithmes qui permettent de trouver la meilleure solution possible, La plupart des problèmes d'optimisation combinatoire multiobjectif sont NP-difficiles, rendant les méthodes exactes très coûteuses en temps de calcul pour les instances de grande taille.
De manière générale, on peut distinguer deux grandes familles d'approches exactes : la séparation et l'évaluation progressive (comme le branch and bound), ainsi que les méthodes de plan de coupe.
La programmation dynamique, bien qu'étant aussi une technique exacte, n'a pas été identifiée dans la littérature sur les problèmes multiobjectifs.

4.3.1 Procédure de séparation et d'évaluation

L'algorithme "Branch and Bound" est une méthode d'énumération et d'évaluation progressive des différentes solutions possibles à un problème. Son principe repose sur deux étapes principales :

_ Séparation :

Elle consiste à diviser l'ensemble des solutions réalisables en sous-ensembles, formant une hiérarchie arborescente. Cette décomposition du problème initial en sous-problèmes plus simples permet de réduire l'espace de recherche. Cependant, la simple séparation serait trop coûteuse en temps de calcul,ce qui nécessite l'intégration d'autres étapes.

_ Evaluation :

L'objectif principal de la phase d'évaluation dans la méthode du branch and bound est de réduire progressivement l'espace de recherche en n'explorant que les nœuds les plus prometteurs. Cela permet d'atteindre plus rapidement la solution optimale.

4.3.2 Les approches non Pareto

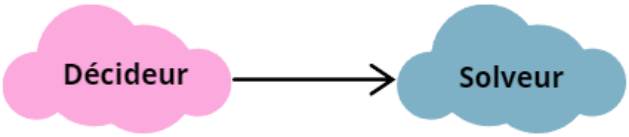
Ces méthodes ne traitent pas le problème comme un véritable problème multiobjectif. Elles réduisent le problème initial à un ou plusieurs problèmes mono-objectifs.

Le décideur peut intervenir en amont de la résolution ,après celle-ci , ou de manière intéractive :

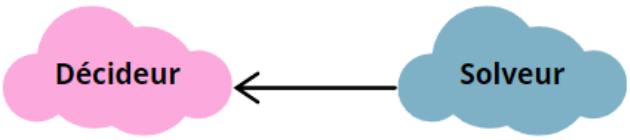
.Les méthodes a préférence à priori

Pour les quelles l'utilisateur définit le compromis qu'il désire réaliser (il fait part de ses préférences) avant de lancer le processus d'optimisation.

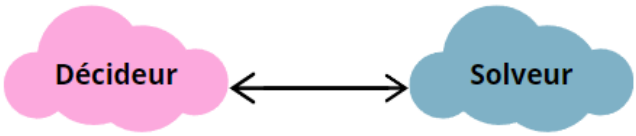
On retrouve dans cette famille la plupart des méthodes par agrégation (où les fonctions objectif sont fusionnées en une seule).



.Les méthodes a préférence à postériori Pour lesquelles l'optimisation est effectuée sans qu'aucun classement des objectifs ne soit initialement spécifié de la part du décideur .
on cherche à lui fournir un ensemble de bonne solutions ensuite il garde les solutions les plus appropriées (convenables).



.Les méthodes a préférence progressive Pour les quelles les processus de décision et d'optimisation sont alternés, permettant au décideur d'orienter la recherche de façon interactive en fonction de ses préférences et des solutions déjà obtenues.



La méthode pondérée (weighted method)
Supposons que le modèle de programmation par objectifs comporte 2 objectifs et que le ième objectif est donné comme suit :

$$\min f_i, i = \{1,2\}$$

la fonction objective combinée est alors définie comme suit :

$$\min \sum_i w_i f_i, i = \{1,2\}$$

(ie)

$$\min w_1 f_1 + w_2 f_2$$

les paramètres $w_i, i = \{1,2\}$ sont des poids positifs qui reflètent la préférence du décideur quant à l'importance relative de chaque objectif.
Par exemple ,$w_i = 1, \forall i$,signifie que les deux objectifs ont le même poids (la détermination des valeurs spécifiques de ces poids est subjective) .
si $\sum_i w_i = 1$alors on obtiendra la la méthode la plus simple parmi les méthodes d'optimisation multi-objectif**Agrégation par pondération**
Méthode ε contrainte
Le principe de la méthode ε contrainte consiste dans le cas bi-objectif à borner l'un des objectifs (en générale le plus difficile à résoudre) et à optimiser l'autre objectif (optimisation mono-objectif) de la la façon suivante :
. Choisir un objectif à optimiser prioritairement.
. Choisir un vecteur de contraintes initiales ε
. Transformer le problème en gardant l'objectif prioritaire et en transformant les autres objectifs en contraintes d'inégalités comme suit :

$$\begin{cases} \min f_k(x) \\ x \in \mathbb{X} \\ s.t \\ f_j(x) \leq \varepsilon_j, j = 1 \ldots n, j \in k \end{cases} \tag{4.1}$$

4.3.3 Méthode approchée

Les méthodes approchées permettent de trouver de manière rapide une solution la plus proche possible de celle d'une méthode exacte, cependant cette solution n'est pas forcément la solution optimale.
Le but d'une heuristique est de trouver une solution qui s'approche autant que possible de celle obtenue par une méthode exacte, mais en moins de temps.
La qualité d'une méthode approchée est évaluée en comparant la solution qu'elle propose à la solution optimale, mesurant ainsi l'écart entre les deux. Plus cette différence est minime, meilleure est l'heuristique.
1.Les heuristiques
Les heuristiques sont catégorisées en différentes classes, chacune déterminée par sa manière de fonctionnement.
Construction VS Amélioration

Une heuristique est qualifiée de **"construction"** lorsqu'elle démarre à partir de l'instance initiale du problème pour construire une solution réalisable. Une heuristique est qualifiée **"d'amélioration"** lorsqu'elle part d'une solution réalisable existante et l'améliore en explorant les solutions voisines.

2.Les métaheuristiques
Solution VS Population

On dit d'une métaheuristique qu'elle est à base de population si elle génère/construit plusieurs solutions.

Les algorithmes génétiques sont des algorithmes qui s'inspirent de la science génétique développée au XIXe siècle par Darwin .Ces approches permettent l'évolution d'une génération (solution)
Étape 1 : Initialisation on choisit n individus qui représente la population initiale à une autre par un ensemble d'opérations.
Étape 2 : **Évaluation** on évalue chaque individu par la fonction objectif.
Étape 3 : **Sélection** on définit les individus de la génération P qui vont être dupliqués dans la nouvelle population.
A chaque génération il y a deux opérateurs de sélection : la sélection de reproduction qui détermine les individus qui vont se reproduire durant une génération, la sélection de remplacement qui détermine quels individus devront disparaître de la population.
Étape 4 : **Reproduction** on utilise des opérateurs génétiques (croisement et mutation) pour produire la nouvelle génération. Les opérateurs de mutation modifient un individu pour en former un autre tandis que les opérateurs de croisement engendrent un ou plusieurs enfants à partir de combinaisons de deux parents.

4.4 Les approches Pareto

Dans les approches Pareto, les objectifs du problème ne sont pas transformés et sont traités de manière équitable et sans privilégier l'un par rapport à l'autre lors de la résolution.

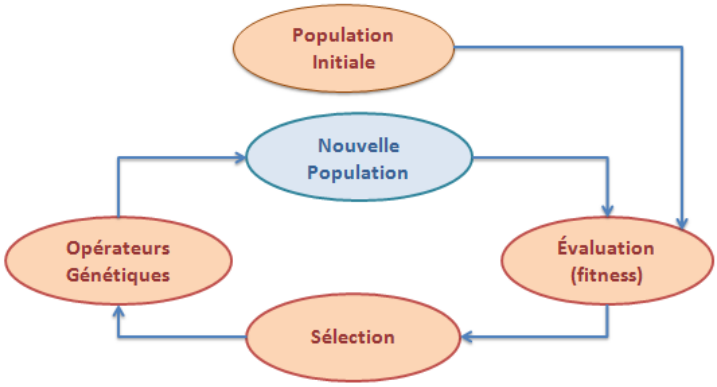


FIGURE 4.1 – Cycle génétique

La dominance au sens de Pareto

.une solution x domine y au sens de Pareto si x est au moins aussi bon pour tout le monde et strictement meilleur pour quelqu'un.
.(i) Si pour tout $i \in I$, $x \succeq y$
.(ii) pour un $i \in I$, $x \succ y$

Pareto optimal

x est dit Pareto optimal, s'il n'existe pas une solution y domine x. Une solution Pareto optimale est connue aussi sous le nom de :
"solution efficace "," non inférieure" ou **"non dominée "**.

Front de Pareto

Le front (frontière) de Pareto est l'ensemble des solutions Pareto optimales qui sont composées des points, ne sont dominés par aucun autre.
Le front de Pareto appelé aussi **surface de compromis** ou **l'ensemble des solutions efficaces** .

Qualité des solutions

.Convergence

.S'approcher du front de Pareto

.Diversité

.Garantir la répartition .

. Obtenir un nombre de solutions suffisant .

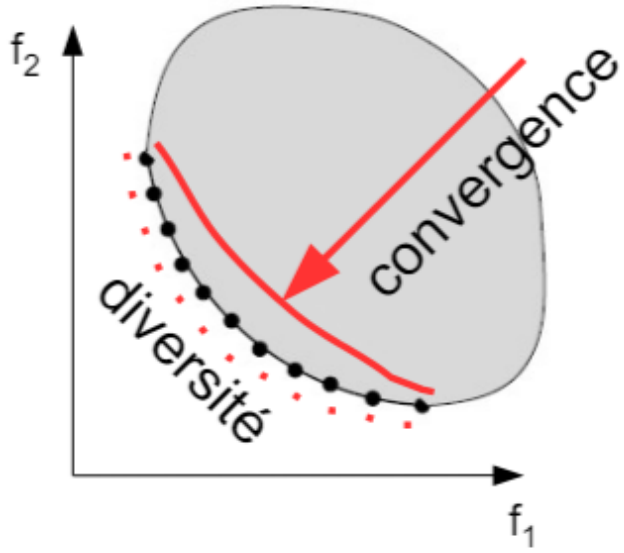


FIGURE 4.2 – Les caractéristiques du front de Pareto

NSGAII

Bien qu'il existe de nombreux algorithmes d'optimisation multi-objectif mais ils ne conservent pas leurs solutions Pareto optimales trouvées au cours des générations.
Nous avons choisi de présenter uniquement le NSGA-II(Non Dominated Sorting Genetic Algorithm II) introduit par deb et al (2002).
Une population de parents (P_t)de taille N et une population d'enfants(C_t)de taille N sont regroupées. .Une population $R_t = (P_t \cup C_t)$ de taille 2N résulte .La population (R_t) est ensuite répartie en plusieurs fronts (F1, F2,...) par une procédure de tri,
Cette méthode est basée sur une classification en plusieurs niveaux des individus. On cherche les individus non dominées ,on les place dans F1 et on l'elimine de l'ensemble des individus. Ensuite ,on refait la même chose avec les autres individus.
Une nouvelle population parent (P_{t+1}) est formée (premier front F1, second front F2, etc.) tant que le nombre d'individus contenus dans ceux-ci ne dépasse pas N.
une procédure de crowding est appliquée sur le premier front suivant si $|(P_{t+1})| \geq N$ Les individus de ce front sont utilisés pour calculer la distance de crowding entre deux solutions Une fois que les individus appartenant à la population (P_{t+1}) sont identifiés, une nouvelle population enfant (C_{t+1}) est

créée par sélection, croisement et mutation.

Exemple

Pour bien comprendre **la notion de tri non dominé** prenons un exemple où un consommateur cherche à acheter un téléphone portable et veut minimiser à la fois le prix d'achat et la consommation d'énergie du téléphone.

Voici trois options de téléphones disponibles sur le marché avec les données correspondantes :

Téléphone	Prix (DA)	Consommation d'énergie (W)
A	6000000	5
B	7000000	4
C	65000000	6

Pour effectuer un tri non dominé, nous devrons identifier les téléphones qui ne peuvent pas être améliorés dans un critère sans détériorer l'autre. En d'autres termes, aucun téléphone ne doit être dominé par un autre dans les deux critères.

Dans cet exemple le téléphone A est moins coûteux que B et C, mais B consomme moins d'énérgie par rapport aux autres. B est préférable à A en consommation d'énérgie et A est préférable à B en prix d'achat donc les téléphones A et B sont non dominés donc ils sont dans le même Front.

Cependant C est dominé par A donc C est classé dans le deuxième front.

Front 1={A,B}

Front 2={C}

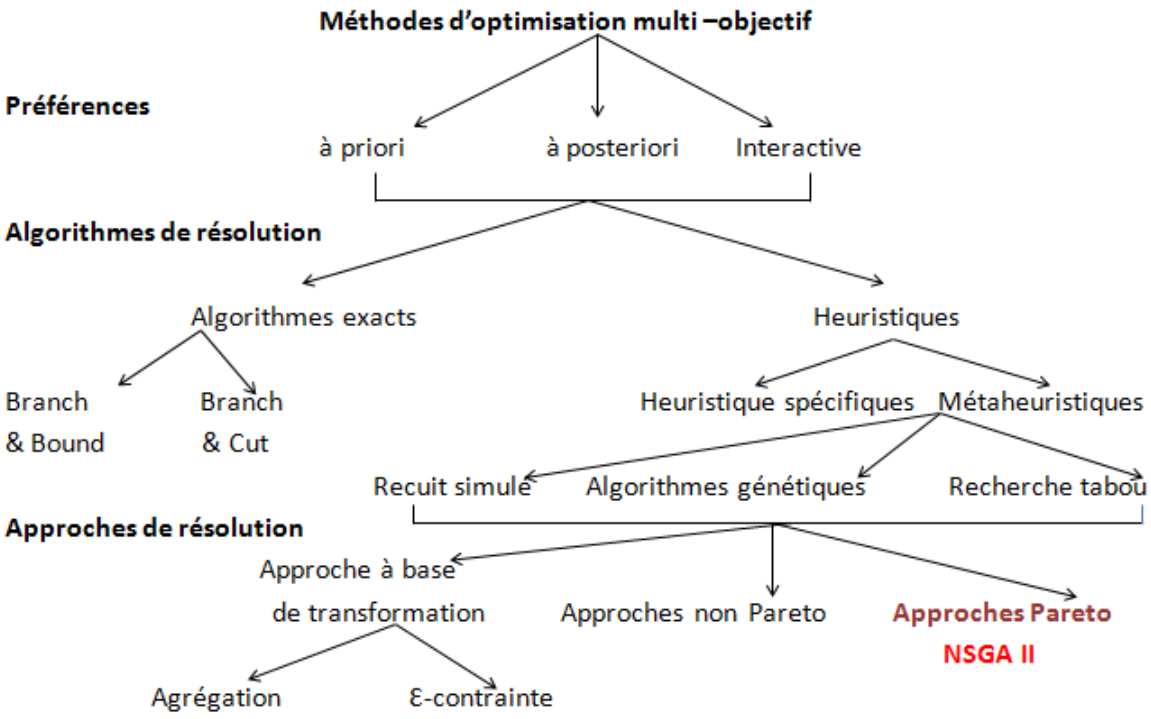


FIGURE 4.3 – schéma récapitulatif des méthodes de résolution des problèmes multi-objectif

Pour plus d'informations sur les méthodes de résolution voir [3],[5],[6],[8],[9],[10],[18],[19]

Chapitre 5

Implémentation

Chapitre 5

5.1 Introduction

Nous aboutissons maintenant à l'étape finale de notre travail, nous décrivons les 3 phases principales de résolution ainsi que l'interprétation des résultats obtenus après l'implémenation.

5.2 Environnement Informatique

Pour la réalisation de notre solution numérique qui est l'objet de ce mémoire,nous avons passé par plusieurs logiciels et outils que nous présenterons dans ce chapitre.

5.3 L'outil Google Earth PRO

Google Earth Pro est la version professionnelle du célèbre logiciel de visualisation et d'exploration géographique Google Earth.
•Données géospatiales riches :
Accès à des millions d'images satellite, de cartes et de données géographiques détaillées.
•Outil d'analyse avancés :
Mesure de distances, calcul de surfaces, planification des projets et modélisation 3D.
•Recherche par nom, adresse ou coordonnées :
positionnerment automatique de la vue sur ce lieu avec précision.

5.4 Les coordonnées géographiques des entrepôts et des stations services

On a utilisé Google Earth Pro pour extraire les coordonnées géographiques associées a chaque entrepôt et chaque stations service y compris les longitudes et les lattitudes.

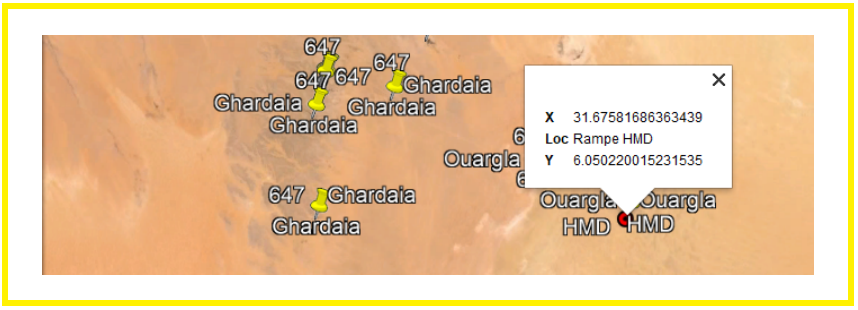


FIGURE 5.1 – Exemple de longitude et latitude (X et Y respectivement) de la rampe HMD

FIGURE 5.2 – Exemple de longitude et latitude d'une station service à Chlef

Les longitudes et les lattitudes sont tous résumés dans les tableaux ci-dessous :
Région SUD

Rampe	Latitude	Longitude
HMD	31.67581686	6.050220015
Adrar	28.21046606	-0.1562585759

TABLE 5.1 – Les rampes du sud avec leurs coordonnées géographiques

Région NORD

Entrepôt Rampe	Latitude	Longitude
Entrepôt Béjaia	36.74895624	5.077816435
Entrepôt El khroub	36.22971597	6.74874158
Entrepôt Berrahal	36.83817637	7.428341359
Entrepôt Skikda	36.88423463	6.924131012
Entrepôt Caroubier	36.73716334	3.126445667
Entrepôt El Harrach	36.7070383	3.140242077
Entrepôt Chiffa	36.46802918	2.735858947
Entrepôt Oran	35.67925146	-0.6219610888
Entrepôt Remchi	35.05465656	-1.422084531
Entrepôt SBA	35.18825186	-0.587997066
Rampe Arzew	35.8306058420	-0.3255389882

TABLE 5.2 – Les rampes et les entrepôts du nord avec leurs coordonnées géographiques

5.5 Algorithme d'Attribution Stations-Entrepôts

Data Scrapping API : est le processus automatisé d'extraction de données à partir de sites web ou de bases de données en ligne on la utiliser pour calculer les distances entre les entrepôts et les stations-service en utilisant des requêtes HTTP pour obtenir les données de distance à partir de Google Maps.

Algorithm 1: Algorithme pour attribuer les stations-services aux entrepôts pour une région donnée

```
Input: Liste des entrepôts et stations-services pour une région
Output: Matrices des entrepôts et leurs stations-services pour cette région
foreach station-service dans la région do
    foreach entrepôt dans la région do
        Calculer la distance entre l'entrepôt et la station-service ;
    end
    Associer la station-service la plus proche à l'entrepôt ;
end
```

Wilaya	station service	Demande totale du GO	Demande totale d'ESP
Laghouat	8	75011	18799
El Oued	5	149011	34118
Biskra	8	97439	25872
Ouled Djellal	1	17497	5905
Bechar	11	35142	12472
Tindouf	6	28951	12828
El bayadh	10	33798	6310
Ghardaia	8	67797	19174
Ouargla	11	81428	22730
Illizi	6	23933	11901
Tam	7	114885	68744
Adrar	7	117086	30575
El Meghaier	1	15102	8486
In Salah	2	42845	4784
In Guezzam	1	5113	3480
Djanet	2	3072	2948
El Meniaa	1	57868	4269

TABLE 5.3 – La demande dans la région SUD

5.6 Approche gloutonne

L'algorithme glouton construit une solution étape par étape en choisissant à chaque étape la solution qui semble la meuillere .

. Nous allons l'intégrer dans notre travail en tant qu'élément fondamental de nos méthodes approximatives.

Choix de l'approche gloutonne :

.**Simplicité**

Les algorithmes gloutons sont simples à comprendre et à mettre en œuvre, ce qui les rend très efficaces.

.**Rapidité**

Leur exécution est généralement rapide et nous permet d'avoir des résultats immédiats.

.**Optimisation locale**

L'obtenition des solutions optimales pour des sous-problèmes, même si la solution globale n'est pas forcément optimale.

Étape 1 : Calculer les rapports capacité/coût pour chaque type de bac et trier ces bacs par ordre décroissant de ce rapport.

Étape 2 : Pour chaque entrepôt installer des bacs en fonction de la demande spécifique de l'entrepôt et du budget restant.

Maximiser l'installation de bacs en choisissant d'abord ceux ayant le meilleur rapport capacité/coût.

Étape 3 : Installer autant de bacs que possible en respectant la demande et le budget disponible, en mettant à jour les capacités installées et le coût total.

Algorithm 2: GreedyAlgorithm

Input : Budget B, Liste des entrepôts E, Liste des bacs L
Output : Capacités des bacs installés et budget restant
Initialisation :
Calculer les rapports $\frac{capacit}{cot}$ pour chaque bac $b \in L$
Trier L par rapport $\frac{capacit}{cot}$ décroissant
InstallerBacs$d$, c, L, B
Input : Demande d, Capacité actuelle c, Liste triée des bacs L, Budget B
Output : Bacs installés, Coût total, Budget restant
$bacs_installes \leftarrow$ Dictionnaire de capacités de bacs initialisé à zéro
$cout_total \leftarrow 0$
$demande_restante \leftarrow d - c$
if $demande_restante \leq 0$ **then**
  return $bacs_installes, cout_total, B$
end if
for chaque $(capacit, \frac{cot}{capacit}, cot) \in L$ **do**
  if $B \leq 0$ **then**
    break
  end if
  $bacs_a_installer \leftarrow \min(\frac{demande_restante}{capacit}, \frac{B}{cot})$
  if $bacs_a_installer > 0$ **then**
    $bacs_installes[capacit] \leftarrow bacs_installes[capacit] + bacs_a_installer$
    $demande_restante \leftarrow demande_restante - bacs_a_installer \times capacit$
    $B \leftarrow B - bacs_a_installer \times cot$
    $cout_total \leftarrow cout_total + bacs_a_installer \times cot$
  end if
end for
return $bacs_installes, cout_total, B$
Installer les bacs dans les entrepôts :
for chaque entrepôt $e \in E$ **do**
  $(bacs_installes_ESP, cout_total_ESP, B) \leftarrow$
  InstallerBacs$e.demande_ESP, e.capacite_ESP, L, B$
  $(bacs_installes_GO, cout_total_GO, B) \leftarrow$
  InstallerBacs$e.demande_GO, e.capacite_GO, L, B$
  $e.installed_bins_ESP \leftarrow bacs_installes_ESP$
  $e.installed_bins_GO \leftarrow bacs_installes_GO$
end for
Affichage des résultats :
Afficher "Budget restant : ", B
Afficher "Entrepôts après installation :"
for chaque entrepôt $e \in E$ **do**
  Afficher $e.name$, "Installed ESP Bin Capacities = ", $e.installed_bins_ESP$
  Afficher $e.name$, "Installed GO Bin Capacities = ", $e.installed_bins_GO$
end for

5.7 Les étapes clés de cet algorithme

• **Étape 1 :** Initialiser le stock initiale de la rampe et son besoin en GO et ESP.
•**Étape 2 :** Comparer le stock actuel de la rampe avec sa demande .
•**Étape 3 :** Transférer la quantité nécessaire de produit à la rampe et mise à jour du stock .

Algorithm 3: Gestion du stock de la rampe

Étape 1 : Initialiser le stock initial de la rampe et son besoin en GO et ESP.
Initialiser $stock_initial_GO$
Initialiser $stock_initial_ESP$
Initialiser $besoin_GO$
Initialiser $besoin_ESP$
Étape 2 : Comparer le stock actuel de la rampe avec sa demande.
if $stock_initial_GO < besoin_GO$ **then**
  $quantit_a_transferer_GO = besoin_GO - stock_initial_GO$
else
  $quantit_a_transferer_GO = 0$
end if
if $stock_initial_ESP < besoin_ESP$ **then**
  $quantit_a_transferer_ESP = besoin_ESP - stock_initial_ESP$
else
  $quantit_a_transferer_ESP = 0$
end if
Étape 3 : Transférer la quantité nécessaire de produit à la rampe et mise à jour du stock.
Transférer $quantit_a_transferer_GO$ de GO vers la rampe
Transférer $quantit_a_transferer_ESP$ d'ESP vers la rampe
Mettre à jour $stock_GO = stock_initial_GO + quantit_a_transferer_GO$
Mettre à jour $stock_ESP = stock_initial_ESP + quantit_a_transferer_ESP$

5.8 Les étapes clés de cet algorithme

• **Étape 1 :**Initialiser un dictionnaire vide *Appro* pour stocker les détails de l'approvisionnement et préparer les données .

•**Étape 2 :** Pour chaque produit demandé, vérifier si une seule raffinerie possède suffisamment de quantité du produit pour satisfaire la demande entière de l'entrepôt.

Si une raffinerie est trouvée avec une quantité suffisante pour les deux produits, sélectionner cette raffinerie comme raffinerie optimale, réduire la quantité disponible dans cette raffinerie, et attribuer la quantité demandée pour les deux produits à cette raffinerie dans le dictionnaire *Appro*.

•**Étape 3 :** Si une raffinerie peut satisfaire entièrement la demande pour un produit, sélectionner cette raffinerie pour approvisionner ce produit et rechercher une autre raffinerie pour le deuxième produit.
Sinon répartir la demande sur toutes les raffineries disponibles.

Pour chaque raffinerie, attribuer une quantité appropriée du produit demandé dans le dictionnaire *Appro* et ajuster la quantité disponible dans cette raffinerie.

•**Étape 4 :**Répéter les étapes 2 et 3 pour chaque produit demandé dans la liste des demandes d'entrepôts.

•**Étape 5 :**Retourner le dictionnaire Appro qui contient la quantité transportée pour chaque produit depuis les raffineries vers les entrepôts.

Algorithm 4: Approvisionnement de la raffinerie vers l'entrepôt

Require: Liste des Capacités raffineries R
Require: Liste des demandes d'entrepôts DE
Ensure: produit : qte approvisionnée [valeur] depuis la raffinerie ["nom"]

```
Étape 1 : Initialiser Appro comme un dictionnaire vide
for all (produit, demande) dans DE do
    qte_demandee ← demande
    raffinerie_optimale ← None
    Étape 2 : Vérifier si une seule raffinerie possède suffisamment de quantité du produit pour satisfaire la demande entière de l'entrepôt
    for all raffinerie dans R do
        capacite_raffinerie ← raffinerie[produit]
        if capacite_raffinerie ≥ qte_demandee then
            raffinerie_optimale ← raffinerie
            Appro[(produit, raffinerie_optimale["nom"])] ← qte_demandee
            raffinerie[produit] ← raffinerie[produit] − qte_demandee {Mise à jour de la qte disponible dans la raffinerie optimale}
            break
        end if
    end for
    if raffinerie_optimale = None then
        Étape 3 : Si une raffinerie peut satisfaire entièrement la demande pour un produit :
        for all raffinerie dans R do
            capacite_raffinerie ← raffinerie[produit]
            if capacite_raffinerie ≥ qte_demandee then
                Appro[(produit, raffinerie["nom"])] ← qte_demandee
                raffinerie[produit] ← raffinerie[produit] − qte_demandee {Mise à jour de la qte disponible dans la raffinerie}
                break
            end if
        end for
        Rechercher une autre raffinerie pour le deuxième produit
    else
        Étape 3 bis : Répartir la demande sur toutes les raffineries disponibles
        for all raffinerie dans R do
            capacite_raffinerie ← raffinerie[produit]
            qte_approvisionnable ← min(capacite_raffinerie, qte_demandee)
            Appro[(produit, raffinerie["nom"])] ← qte_approvisionnable
            raffinerie[produit] ← raffinerie[produit] − qte_approvisionnable {Mise à jour de la qte disponible dans la raffinerie}
            qte_demandee ← qte_demandee − qte_approvisionnable
            if qte_demandee ≤ 0 then
                break
            end if
        end for
    end if
end for
return Appro
```

Étape 4 : Conversion de $Appro$ en matrice
Déterminez le nombre de raffineries et d'entrepôts
Initialisez une matrice avec des zéros
for all élément $(produit, raffinerie)$ dans $Appro$ **do**
    $i \leftarrow$ indice de la raffinerie dans la liste des raffineries
    $j \leftarrow$ indice de l'entrepôt dans la liste des entrepôts
    Mettre à jour la valeur de la cellule (i, j) de la matrice avec la quantité approvisionnée
end for

```
Output                                                    Clear
Quantité de GO transportée de Raffinerie_HMD vers la rampe HMD: 26575.52
Quantité de GO transportée de Raffinerie_Adrar vers la rampe Adrar: 12434.35
Quantité de ESP transportée de Raffinerie_HMD vers la rampe HMD: 8156.67
Quantité de ESP transportée de Raffinerie_Adrar vers la rampe Adrar: 3219.31
```

FIGURE 5.3 – solution initiale d'approvisionnment

5.9 Perturabation de la solution

Une perturbation d'une solution est un changement ou une modification d'une solution existante au niveau des paramètres , des conditions initiales ou des entrées .
Perturbation par critére pour la couverture des stations services :
1.critére quantité
•Identifier les stations services similiaires qui ont la même demande en GO ou ESP et permuter entre eux.
•Affecter les stations services aux entrepôts .
2.critére distance
• Pour chaque entrepôt voir s'il y a des stations services qui sont couvertes par l'autre entrepôt et qui ne sont pas trop loin de cet entrepôt, On calcule la distance entre la station service et les entrepôts ensuite on compare la différence entre ces distances (la distance avec l'entrepôt qui la couvre et un autre entrepôt) si elle est au plus 50 km la station service sera reassignée. En fait, l'idée est de trouver les stations services qui sont au milieu et de les réaffecter.
3.critére distance-quantité

• Chercher les stations services jumelles dans les entrepôts celles qui sont semblables en demandes et en distance.

Perturbation Par remplacement des bacs :

•Séléctionnez les bacs (installés) de $20000m^3$ à remplacer de chaque entrepôt.

•Trouvez les bacs qui ont une capacité inférieure à la capacité du bac Séléctionné.

•Cherchez toutes les combinaisons possibles de ces bacs.

•Si la capacité totale de chaque combinaison est inférieure ou égale à la capacité du bac choisis ,alors remplacez le.

• Répétez les étapes prcédente pour chaque produit et pour chaque centre primaire.

• Retournez les nouvelles solutions perturbées.

Perturbation Par permutation pour l'approvisionnement :

•Identifier les paires d'entrepôt qui sont approvisionnés par une seule raffinerie pour les deux produits.

• Pour chaque paire d'entrepôts :

Si les deux raffineries peuvent répondre aux demandes des entrepôts échangé (pour GO ou ESP)alors permuter entre eux.

•Retourner les nouvelles solutions obtenues après chaque échange.

5.10 Python

Python est un langage de programmation de haut niveau, interprété, développé par Guido Von Rossum en 1989 à l'université d'Amesterdam.

FIGURE 5.4 – Python Logo

Justification de choix de langage
.Simplicité de syntaxe : sa syntaxe qui est simple et concise, facilitant l'apprentissage.
.Multiplateforme : son exécution sur de nombreux systèmes d'exploitation, rendant le développement portable.
.Polyvalence :son utilisation dans une grande variété de domaines, de l'automatisation à l'IA.
.Diversité des bibliothèques :l'offre d'une vaste bibliothèque de fonctions intégrées, facilitant la résolution de problèmes complexes.

FIGURE 5.5 – Le classement des langages de programmation en 2024

En résumé Python est un langage de programmation puissant qui s'est imposé comme l'un des langages les plus populaires au monde. Avec une syntaxe lisible et une grande flexibilité, Python permet de développer rapidement des applications, des solutions innovantes et des logiciels.

5.11 Les bibliothèques utilisées

Parmi les packages prédéfini en Python nous avons utilisé :
Numpy :(abréviation de Numerical Python) une bibliothèque qui propose une structure de données populaire, les tableaux (de type array), sur lesquels il est possible d'effectuer des calculs.

requests : une bibliothèque qui facilite l'interaction avec des API web et la récupération de données à partir de serveurs web.
Pandas : :une bibliothèque qui prend en charge la lecture et l'écriture de données à partir de divers formats de fichier, tels que CSV, Excel, SQL, JSON,...
Json :une bibliothèque qui lit les données depuis un fichier JSON et les convertit en structures de données Python manipulables.
re : une bibliothèque qui est utilisée pour les expressions régulières.
itertools : un module qui traite les données de manière itérative et combinatoire.
tabulate : un module qui formate les données tabulaires en plusieurs formats de sortie, y compris les tables ASCII simples.
Les solutions initiales :Les solutions z_{ikp} sont dans un fichier Excel.

$$x^1_{ijp} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 32 & 0 & 0 & 0 \end{pmatrix} \ , \begin{pmatrix} 17 & 1 & 0 & 0 \\ 3 & 1 & 0 & 0 \end{pmatrix}\right]$$

$$x^2_{ijp} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 64 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 34 & 0 \\ 0 & 1 & 6 & 0 \end{pmatrix}\right]$$

$$x^3_{ijp} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 32 & 32 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 17 & 17 \\ 0 & 1 & 3 & 3 \end{pmatrix}\right]$$

$$x^4_{ijp} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 64 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 0 & 34 \\ 0 & 1 & 0 & 6 \end{pmatrix}\right]$$

$$x^5_{ijp} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 64 & 0 \end{pmatrix} \ , \begin{pmatrix} 17 & 1 & 0 & 0 \\ 3 & 1 & 0 & 0 \end{pmatrix}\right]$$

$$x^6_{ijp} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 32 & 0 & 0 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 34 & 0 \\ 0 & 1 & 6 & 0 \end{pmatrix}\right]$$

$$x^7_{ijp} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 32 & 32 \end{pmatrix} \ , \begin{pmatrix} 17 & 1 & 0 & 0 \\ 3 & 1 & 0 & 0 \end{pmatrix}\right]$$

$$x_{ijp}^{8} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 32 & 0 & 0 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 17 & 17 \\ 0 & 1 & 3 & 3 \end{pmatrix} \right]$$

$$x_{ijp}^{9} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 64 \end{pmatrix} \ , \begin{pmatrix} 17 & 1 & 0 & 0 \\ 3 & 1 & 0 & 0 \end{pmatrix} \right]$$

$$x_{ijp}^{10} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 32 & 0 & 0 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 0 & 34 \\ 0 & 1 & 0 & 6 \end{pmatrix} \right]$$

$$x_{ijp}^{11} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 64 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 17 & 17 \\ 0 & 1 & 3 & 3 \end{pmatrix} \right]$$

$$x_{ijp}^{12} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 32 & 32 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 34 & 0 \\ 0 & 1 & 6 & 0 \end{pmatrix} \right]$$

$$x_{ijp}^{13} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 64 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 0 & 34 \\ 0 & 1 & 0 & 6 \end{pmatrix} \right]$$

$$x_{ijp}^{14} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 64 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 17 & 17 \\ 0 & 1 & 3 & 3 \end{pmatrix} \right]$$

$$x_{ijp}^{15} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 64 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 17 & 17 \\ 0 & 1 & 3 & 3 \end{pmatrix} \right]$$

$$x_{ijp}^{16} = \left[\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 32 & 32 \end{pmatrix} \ , \begin{pmatrix} 0 & 1 & 0 & 34 \\ 0 & 1 & 0 & 6 \end{pmatrix} \right]$$

$$y_{rip}^{1} = \left[\begin{pmatrix} 26575.16 & 8156.67 \\ 0 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 0 \\ 12434,35 & 3219.31 \end{pmatrix} \right]$$

$$y_{rip}^{2} = \left[\begin{pmatrix} 26575.16 & 8156.67 \\ 0 & 0 \end{pmatrix} \ , \begin{pmatrix} 12434,35 & 3219.31 \\ 0 & 0 \end{pmatrix} \right]$$

$$y_{rip}^{3} = \left[\begin{pmatrix} 26575.16 & 8156.67 \\ 0 & 0 \end{pmatrix} \ , \begin{pmatrix} 0 & 3219.31 \\ 12434,35 & 0 \end{pmatrix} \right]$$

5.12 Autres Méthodes proposées

L'automatisation des entrepôts
Développer un système intégré en utilisant des capteurs IOT pour surveiller le stock d'un entrepôt en temps réel et automatiser la communication avec les raffineries pour le réapprovisionnemnet
Approvisionnement
1.Méthode multi-agents
Agents :
- Raffinerie
- Entrepôt

Messages :
- DemandeApprovisionnement(produit, quantité).
- Approvisionnement(produit, quantité, raffinerie).

Algorithme principal :
1. Initialiser les agents raffinerie et entrepôt avec leurs stocks respectifs.
2. Pour chaque demande de l'entrepôt :
a. L'entrepôt envoie une DemandeApprovisionnement à toutes les raffineries.
b. Chaque raffinerie reçoit la demande et vérifie si elle peut satisfaire la demande.
c. Si une raffinerie peut satisfaire la demande, elle envoie un Approvisionnement à l'entrepôt avec la quantité disponible et son propre identifiant.
d. L'entrepôt reçoit les réponses des raffineries, sélectionne le meilleure offre en fonction des critères prédéfinis (par exemple, la distance), et met à jour son stock en conséquence.
3. Répéter les étapes 2 pour chaque demande de l'entrepôt jusqu'à ce que toutes les demandes soient satisfaites.
4. Retourner les détails de l'approvisionnement.

Agent : Raffinerie
- Initialiser avec un stock initial de produits.
- Recevoir une DemandeApprovisionnement de l'entrepôt.
- Vérifier si elle peut satisfaire la demande en fonction de son stock.
- Si oui, envoyer un Approvisionnement à l'entrepôt avec la quantité disponible et son propre identifiant.

Agent :Entrepôt
- Initialiser avec une liste de demandes de produits.
- Envoyer une DemandeApprovisionnement à toutes les raffineries pour chaque produit demandé.
- Recevoir les réponses des raffineries avec les offres d'approvisionnement.
- Sélectionner la meilleure offre en fonction des critères prédéfinis.
- Mettre à jour son stock en conséquence.
but :
exploiter la capacité collective des agents(raffineries) pour résoudre des problèmes qui dépassent les capacités d'un agent individuel .
2.Équilibrage de charge
Données d'entrée :

- Ensemble des raffineries $R = r_1, r_2, ..., r_n$
- Ensemble des rampes $E = e_1, e_2, ..., e_m$
- Capacité de production de chaque raffinerie r : Cr
- Demande de chaque rampe e : De

Procédure :

1. Initialisation :
- Créer un tableau d'allocation A de taille |R| x |E|, initialisé à 0.
- Créer un tableau des capacités résiduelles Cr' = Cr pour chaque raffinerie r.

2. Répartition de la charge :
- Trier les entrepôts par ordre décroissant de demande De.
- Pour chaque entrepôt e dans l'ordre : - Trouver la raffinerie r avec la plus grande capacité résiduelle Cr' capable de satisfaire la demande De.
- Allouer la demande De à la raffinerie r : A[r][e] = De, Cr' = Cr' - De.
- Si la demande De ne peut pas être entièrement satisfaite par r, allouer le reste aux autres raffineries par ordre de capacité résiduelle.

3. Équilibrage de la charge :
- Tant que le système n'est pas équilibré : - Trouver la raffinerie r avec la plus grande capacité résiduelle Cr'.
- Trouver la rampe e avec la plus faible allocation A[r][e].
- Transférer une partie de la charge de e vers d'autres raffineries ayant une capacité résiduelle suffisante.
- Mettre à jour les allocations A et les capacités résiduelles Cr'.

4. Vérification
- Vérifier que toutes les demandes De sont satisfaites.

Cet algorithme vise à répartir de manière optimale la charge de production entre les raffineries en fonction de leurs capacités et des demandes des entrepôts.

P-NSGA II (parametrized NSGA-II)

utilise le surclassement pour favoriser certaines solutions par rapport à d'autres. Cela peut être réalisé en modifiant les probabilités de sélection en fonction des critères (rang de non-dominance, la distance de foule).

Conclusion générale

Notre principale contribution peut être synthétisé de la manière suivante :
.L'analyse de la situation existante
.La proposition d'un modèle mathématique pour le problème d'emplacement des bacs de stockage, ce modèle basé sur la programmation linéaire étant NP-difficile nous avons utilisé les méthodes approximatives **(les heuristiques glouton et plus proche voisin pour la construction de la solution initiale)**ensuite la méta-heuristique NSGA II un outil puissant pour résoudre les problèmes bi-objectif en trouvant un compromis entre les deux objectifs.

Pour conclure, le stage que nous avons effectué au sein de l'entreprise NAFTAL pour la réalisation de ce travail, nous a permis de développer des nouvelles connaissances, mettre en épreuve tout ce qu'on appris durant notre cursus, allant de la compréhension, la modélisation conceptuelle et mathématique jusqu'au codage informatique.

Bibliographie

[1] Dahou, Nour El Yakine. Optimisation de schéma de distribution des carburants à l'horizon 2025. Université d'Alger 1 Benyoucef-Benkhedda, Alger Centre, 2020-2021.

[2] Taha, Hamdy A. Operations Research : An Introduction. 2013.

[3] Collette, Yann and Siarry, Patrick. Optimisation multiobjectif. 2002.

[4] Dhaenens, Clarisse. Optimisation Combinatoire Multi-Objectif : Apport des Méthodes Coopératives et Contribution à l'Extraction de Connaissances. Université des Sciences et Technologie de Lille - Lille I, 2005.

[5] Dahou, Mohamed Lamine et Chetbani, Aissa. Méthodes d'optimisation dans les réseaux de transport et applications. Université de Béjaia, 2017-2018.

[6] Alaya, Inès. Optimisation Multi-Objectif par Colonies de Fourmis : Cas des Problèmes de Sac à Dos. Université Claude Bernard - Lyon I ; Université de la Manouba (Tunisie), 2009.

[7] Konan, Yao Silvère. Cours de Recherche Opérationnelle. Université Félix Houphouët-Boigny.

[8] Irisarri, François-Xavier. Stratégies de Calcul pour l'Optimisation Multiobjectif des Structures Composites. Université Paul Sabatier - Toulouse III, 2009.

[9] Founding Series. International Series in Operations Research Management Science. 2017.

[10] Knowles, Joshua, Corne, David, et Deb, Kalyanmoy. Multiobjective Problem Solving from Nature : From Concepts to Applications. 2007.

[11] Haddadi, Salim. Programmation Linéaire : Une Approche Mathématique et Algorithmique. 2021.

[12] Drira, Nawel. Université Paris-Est ; École nationale d'ingénieurs de Gabès (Tunisie), 2019.

[13] Naftal. Accueil. n.d. Accessed : 2024-04-12.

[14] Ministère de l'Énergie de l'Algérie. Site officiel du Ministère de l'Énergie de l'Algérie, 2024. Accédé : 27-04-2024.

[15] Vajda, Steven. Théorie des Jeux et Programmation Linéaire. 1968.

[16] Cerf, Max. Techniques d'Optimisation - Tome 2 : Optimisation Discrète et Fonctionnelle. 2022.

[17] Churchman, C. West, Ackoff, Russell L., et Arnoff, E. Leonard. Introduction to Operations Research. 1957.

[18] Gendreau, Michel, et Potvin, Jean-Yves (Eds.). Handbook of Metaheuristics. 2010.

[19] Belaidi, Idir, Mohammedi, Kamal, et Brachemi, Belaid. CFM 2009 - 19ème Congrès Français de Mécanique. 2009.

Annexes

```python
# Définition des données
# Liste des entrepôts
entrepots = ['Adrar', 'Hmd']
# Demande mensuelle pour les produits ESP et GO dans les entrepôts
demande_ESP_mensuelle = [95356.50, 797265.45]
demande_GO_mensuelle = [373030.517, 244500.88]
# Capacités actuelles pour les entrepôts
capacite_actuelle_ESP = [16291.06, 152841]
capacite_actuelle_GO = [16847.17, 318792.62]
# Définir la capacité des bacs et leur coût
cap_bac = [8000, 10000, 15000, 20000]
cout_bac = [46962312 ,58702890,83626221,88054335]
# Budget global
budget_global = 721059000000
# Précalcul des rapports capacité/coût pour les bacs et tri
rapports_capacite_cout = [(cap_bac[i], cout_bac[i] / cap_bac[i],
    cout_bac[i]) for i in range(len(cap_bac))]
rapports_capacite_cout.sort(key=lambda x: x[1])
# Fonction pour installer le maximum de bacs avec le minimum de coût
    pour atteindre la demande dans chaque entrepôt
def installer_bacs(demande, capacite_actuelle,
    rapports_capacite_cout, budget):
    bacs_installes = {cap: 0 for cap, _, _ in
        rapports_capacite_cout}
    cout_total = 0
    demande_restante = demande - capacite_actuelle
    if demande_restante <= 0:
        return bacs_installes, cout_total, budget
    for cap, _, cout in rapports_capacite_cout:
        if budget <= 0:
            break
```

```python
        # Nombre de bacs pouvant être installés avec le budget restant et la demande restante
        bacs_a_installer = min(demande_restante // cap, budget // cout)
        if bacs_a_installer > 0:
            bacs_installes[cap] += bacs_a_installer
            demande_restante -= bacs_a_installer * cap
            budget -= bacs_a_installer * cout
            cout_total += bacs_a_installer * cout
    return bacs_installes, cout_total, budget
# Initialisation des structures pour stocker les résultats
bacs_installes_entrepots_ESP = {}
cout_total_entrepots_ESP = {}
bacs_installes_entrepots_GO = {}
cout_total_entrepots_GO = {}
# Variables pour suivre le budget global restant
budget_restants = budget_global
# Installation des bacs dans les entrepôts  pour les deux produits
for entrepot, demande_ESP, demande_GO, capacite_ESP, capacite_GO in zip(entrepots, demande_ESP_mensuelle, demande_GO_mensuelle, capacite_actuelle_ESP, capacite_actuelle_GO):
    bacs_installes_ESP, cout_total_ESP, budget_restants = installer_bacs(demande_ESP, capacite_ESP, rapports_capacite_cout, budget_restants)
    bacs_installes_GO, cout_total_GO, budget_restants = installer_bacs(demande_GO, capacite_GO, rapports_capacite_cout, budget_restants)
    bacs_installes_entrepots_ESP[entrepot] = bacs_installes_ESP
    cout_total_entrepots_ESP[entrepot] = cout_total_ESP
    bacs_installes_entrepots_GO[entrepot] = bacs_installes_GO
    cout_total_entrepots_GO[entrepot] = cout_total_GO
# Affichage des résultats
print("Répartition des bacs par entrepôt pour le produit ESP :")
for entrepot in entrepots:
    print(f"- Entrepôt {entrepot} (ESP):")
    bacs_installes = bacs_installes_entrepots_ESP[entrepot]
    cout_total = cout_total_entrepots_ESP[entrepot]
    for cap, count in bacs_installes.items():
        print(f"  - {count} bacs de capacité {cap}")
    print(f"Coût total : {cout_total}")
print("\nRépartition des bacs par entrepôt pour le produit GO :")
for entrepot in entrepots:
    print(f"- Entrepôt {entrepot} (GO):")
    bacs_installes = bacs_installes_entrepots_GO[entrepot]
    cout_total = cout_total_entrepots_GO[entrepot]
    for cap, count in bacs_installes.items():
        print(f"  - {count} bacs de capacité {cap}")
    print(f"Coût total : {cout_total}")
print(f"\nBudget global restant : {budget_restants}")
```

FIGURE 5.6 – code d'installation des bacs

```python
# Initialisation des rampes avec leurs besoins et la raffinerie rattachée pour les deux produits (GO et ESP)
rampe_HMD = {
    "stock": {"GO": 0, "ESP": 0},
    "besoin": {"GO": 26575.52, "ESP": 8156.67},
    "raffinerie": {
        "nom": "Raffinerie_HMD",
        "capacite_production": {"GO": 318792.61, "ESP": 61136.75}
    }
}
rampe_Adrar = {
    "stock": {"GO": 0, "ESP": 0},
    "besoin": {"GO": 12434.35, "ESP": 3219.31},
    "raffinerie": {
        "nom": "Raffinerie_Adrar",
        "capacite_production": {"GO": 13300, "ESP": 6516.42}
    }
}
# Fonction pour vérifier si la rampe HMD a besoin de produit
def rampe_HMD_a_besoin_de_produit(produit):
    return rampe_HMD["stock"][produit] <rampe_HMD["besoin"][produit]
# Fonction pour vérifier si la rampe Adrar a besoin de produit (GO ou ESP)
def rampe_Adrar_a_besoin_de_produit(produit):
    return rampe_Adrar["stock"][produit] < rampe_Adrar["besoin"][produit]
# Fonction pour provisionner la rampe HMD à partir de la raffinerie (GO ou ESP)
def provisionner_rampe_HMD(produit):
    quantite_besoin = rampe_HMD["besoin"][produit] - rampe_HMD["stock"][produit]
    capacite = rampe_HMD["raffinerie"]["capacite_production"][produit]
    quantite_transportee = min(quantite_besoin, capacite)
    rampe_HMD["stock"][produit] += quantite_transportee
    print(f"Quantité de {produit} transportée de {rampe_HMD['raffinerie']['nom']} vers la rampe HMD: {quantite_transportee}")
# Fonction pour provisionner la rampe Adrar à partir de la raffinerie (GO ou ESP)
def provisionner_rampe_Adrar(produit):
    quantite_besoin = rampe_Adrar["besoin"][produit] - rampe_Adrar["stock"][produit]
    capacite = rampe_Adrar["raffinerie"]["capacite_production"][produit]
    quantite_transportee = min(quantite_besoin, capacite)
    rampe_Adrar["stock"][produit] += quantite_transportee
    print(f"Quantité de {produit} transportée de {rampe_Adrar['raffinerie']['nom']} vers la rampe Adrar: {quantite_transportee}")

# Exécution de l'algorithme pour chaque produit à chaque rampe
produits = ["GO", "ESP"]
for produit in produits:
    if rampe_HMD_a_besoin_de_produit(produit):
        provisionner_rampe_HMD(produit)

    if rampe_Adrar_a_besoin_de_produit(produit):
        provisionner_rampe_Adrar(produit)
```

FIGURE 5.7 – Code d'approvisionnement

```python
import requests
from concurrent.futures import ThreadPoolExecutor
from time import time as timer
counter = 0

def calculate_distance(lat1, lon1, lat2, lon2):
    try:
        global counter
        url = f"http://router.project-osrm.org/route/v1/driving/{lon1},{lat1};{lon2},{lat2}?overview=false"
        response = requests.get(url, timeout=5)
        data = response.json()
        distance = data['routes'][0]['distance']
        return distance
    except Exception as ex:
        pass


def stations_couvertes_par_entrepot(entrepots, stations):
    couvertes = {entrepot['id']: [] for entrepot in entrepots}
```

```python
    with ThreadPoolExecutor(2) as executor:
        futures = []
        for station in stations:
            futures.append(executor.submit(find_nearest_entrepot, station, entrepots))
        for future in futures:
            plus_proche_entrepot, station_id = future.result()
            couvertes[plus_proche_entrepot].append(station_id)
    return couvertes
def find_nearest_entrepot(station, entrepots):
    plus_proche_entrepot = None
    distance_min = float('inf')
    for entrepot in entrepots:
        distance = calculate_distance(station['lat'], station['lon'], entrepot['lat'], entrepot['lon'])
        if distance < distance_min:
            distance_min = distance
            plus_proche_entrepot = entrepot['id']
    return plus_proche_entrepot, station['id']
```

```python
# Exemple d'entrepôts et de stations-service
# Example lists of warehouses and stations for the South region
entrepots = [
    {'id': 1, 'lat': 31.675816863, 'lon': 6.050220015231535},  # Example coordinates
    {'id': 2, 'lat': 28.210466064, 'lon': -0.15625857592481263}
]

stations = [
    {'id': 1, 'lat': 34.108282, 'lon': 2.090939},
    {'id': 2, 'lat': 34.153213, 'lon': 2.169662},
    {'id': 3, 'lat': 33.816861, 'lon': 2.871409},
    {'id': 4, 'lat': 33.787712, 'lon': 3.139823},
    {'id': 5, 'lat': 32.947543, 'lon': 3.256372},
    {'id': 6, 'lat': 33.816909337960546, 'lon': 2.87135973572731},
    {'id': 7, 'lat': 33.81734, 'lon': 2.871945},
    {'id': 8, 'lat': 33.816959, 'lon': 2.872611},
    {'id': 9, 'lat': 33.38742496745898, 'lon': 6.83711439371109},
    {'id': 10, 'lat': 33.50505190167835, 'lon': 6.797824058885453},
    {'id': 11, 'lat': 33.54317626032765, 'lon': 5.988550636466243},
    {'id': 12, 'lat': 34.262787169930455, 'lon': 5.92054322722106},
    {'id': 13, 'lat': 33.94668274728171, 'lon': 5.915790796498186},
```

```
61      {'id': 14, 'lat': 34.87989558284085, 'lon': 5.714446306228638},
62      {'id': 15, 'lat': 34.7994444, 'lon': 5.715277777777778},
63      {'id': 16, 'lat': 34.7644444, 'lon': 5.8983333333333325},
64      {'id': 17, 'lat': 34.6844444, 'lon': 6.495833333333334},
65      {'id': 18, 'lat': 35.1983333, 'lon': 5.693888888888889},
66      {'id': 19, 'lat': 34.73039736795763, 'lon': 5.399424884672044},
67      {'id': 20, 'lat': 34.85275283845618, 'lon': 5.73119103883073},
68      {'id': 21, 'lat': 34.4405556, 'lon': 5.070555555555555},
69      {'id': 22, 'lat': 31.023230428945993, 'lon': -2
            .712673544883728},
70      {'id': 23, 'lat': 32.0528415027339, 'lon': -1.2543685734272003}
            ,
71      {'id': 24, 'lat': 31.62179208225093, 'lon': -2
            .2149159014225006},
72      {'id': 25, 'lat': 29.454760415281555, 'lon': -1
            .440594345331192},
73      {'id': 26, 'lat': 30.1222259207458, 'lon': -2.1791204810142517}
            ,
74      {'id': 27, 'lat': 31.023230428945993, 'lon': -2
            .712673544883728},
75      {'id': 28, 'lat': 32.0528415027339, 'lon': -1.2543685734272003}
            ,
```

```
76      {'id': 29, 'lat': 31.62179208225093, 'lon': -2
            .2149159014225006},
77      {'id': 30, 'lat': 29.454760415281555, 'lon': -1
            .440594345331192},
78      {'id': 31, 'lat': 30.1222259207458, 'lon': -2.1791204810142517}
            ,
79      {'id': 32, 'lat': 30.920761, 'lon': -2.037575},
80      {'id': 33, 'lat': 27.677778121196717, 'lon': -8
            .149382472038269},
81      {'id': 34, 'lat': 28.62454256184381, 'lon': -6.981934905052185}
            ,
82      {'id': 35, 'lat': 29.185004, 'lon': -5.081991},
83      {'id': 36, 'lat': 27.677778121196717, 'lon': -8
            .149382472038269},
84      {'id': 37, 'lat': 28.62454256184381, 'lon': -6.981934905052185}
            ,
85      {'id': 38, 'lat': 29.185004, 'lon': -5.081991},
86      {'id': 39, 'lat': 33.690068, 'lon': 1.021166},
87      {'id': 40, 'lat': 34.051117, 'lon': 0.085402},
88      {'id': 41, 'lat': 33.725783, 'lon': 1.530222},
89      {'id': 42, 'lat': 34.017807, 'lon': 0.923531},
90      {'id': 43, 'lat': 32.911334, 'lon': 0.545829},
91      {'id': 44, 'lat': 33.690068, 'lon': 1.021166},
92      {'id': 45, 'lat': 34.051117, 'lon': 0.085402},
93      {'id': 46, 'lat': 33.725783, 'lon': 1.530222},
94      {'id': 47, 'lat': 34.017807, 'lon': 0.923531},
95      {'id': 48, 'lat': 32.911334, 'lon': 0.545829},
96      {'id': 49, 'lat': 31.693265, 'lon': 6.055094},
97      {'id': 50, 'lat': 32.610884, 'lon': 5.518396},
98      {'id': 51, 'lat': 33.12993, 'lon': 6.06492},
99      {'id': 52, 'lat': 31.695072, 'lon': 6.054493},
100     {'id': 53, 'lat': 31.704164, 'lon': 6.054752},
101     {'id': 54, 'lat': 30.516872, 'lon': 6.462547},
102     {'id': 55, 'lat': 31.687468, 'lon': 9.218609},
103     {'id': 56, 'lat': 31.942248, 'lon': 5.315611},
104     {'id': 57, 'lat': 33.107603, 'lon': 6.085405},
105     {'id': 58, 'lat': 33.129933, 'lon': 6.064922},
106     {'id': 59, 'lat': 31.955257, 'lon': 5.339268},
107     {'id': 60, 'lat': 28.689731, 'lon': 6.504838},
108     {'id': 61, 'lat': 28.684809, 'lon': 8.893593},
109     {'id': 62, 'lat': 29.964623, 'lon': 9.418840},
110     {'id': 63, 'lat': 28.037653, 'lon': 9.564547},
111     {'id': 64, 'lat': 26.518919, 'lon': 8.480516},
```

```
112         {'id': 65, 'lat': 22.795819, 'lon': 5.51732},
113         {'id': 66, 'lat': 22.771438, 'lon': 5.526632},
114         {'id': 67, 'lat': 22.657060, 'lon': 4.570354},
115         {'id': 68, 'lat': 24.020956, 'lon': 5.081959},
116         {'id': 69, 'lat': 19.564612, 'lon': 5.77229},
117         {'id': 70, 'lat': 27.204984, 'lon': 2.487504},
118         {'id': 71, 'lat': 25.257721, 'lon': 3.772238},
119         {'id': 72, 'lat': 24.512893, 'lon': 9.492587},
120         {'id': 73, 'lat': 27.878772, 'lon': -0.289516},
121         {'id': 74, 'lat': 26.730998, 'lon': 0.154578},
122         {'id': 75, 'lat': 26.978782, 'lon': 1.07994},
123         {'id': 76, 'lat': 29.254756, 'lon': 0.231258},
124         {'id': 77, 'lat': 20.444727, 'lon': 1.804394},
125         {'id': 78, 'lat': 27.873752, 'lon': -0.279050},
126         {'id': 79, 'lat': 21.333373, 'lon': 0.954348},
127         {'id': 80, 'lat': 30.559309, 'lon': 2.896769},
128         {'id': 81, 'lat': 32.2577, 'lon': 3.625546},
129         {'id': 82, 'lat': 32.781655, 'lon': 4.477196},
130         {'id': 83, 'lat': 32.391649, 'lon': 4.225551},
131         {'id': 84, 'lat': 31.606393, 'lon': 3.680071},
132         {'id': 85, 'lat': 32.488011, 'lon': 3.696314},
133         {'id': 86, 'lat': 32.477832, 'lon': 3.685695},
134         {'id': 87, 'lat': 32.494318, 'lon': 3.682968},
135         {'id': 88, 'lat': 33.260551, 'lon': -0.316335},
136         {'id': 89, 'lat': 33.717500, 'lon': -0.728080},
137         {'id': 90, 'lat': 32.763631, 'lon': -0.597312},
138         {'id': 91, 'lat': 33.552477, 'lon': -0.264604},
139         {'id': 92, 'lat': 32.75769, 'lon': -0.575473},
140         {'id': 93, 'lat': 33.260551, 'lon': -0.316335},
141         {'id': 94, 'lat': 33.717500, 'lon': -0.728080},
142         {'id': 95, 'lat': 32.763631, 'lon': -0.597312},
143         {'id': 96, 'lat': 33.552477, 'lon': -0.264604}
144     ]
145
146     # Trouver les stations couvertes par chaque entrepôt
147     stations_couvertes = stations_couvertes_par_entrepot(entrepots,
            stations)
148     print("Stations couvertes par chaque entrepôt:")
149     for entrepot, stations in stations_couvertes.items():
150         print(f"Entrepôt {entrepot}: {stations}")
```

FIGURE 5.8 – Code d'attribution stations-entrepôts